DigiCom Notes

Alessandro Vulcu

November 26, 2025

Contents

Chapter 1

Week 13-19 October

1.1 Introduction

In most of the course we'll handle wireless communications, as they're much more difficult than wired communications. In this course we'll stay at the physical layer. We'll see:

- **Channel models**: taking radio communications and making a mathematical model for them. We'll find an accurate model and then we'll apply a tradeoff to it, simplifying it so that it is manageable.
- **OFDM**: Orthogonal Frequency Division Multiplexing.
- **MIMO**: we have multiple antennas at the transmitter and multiple antennas at the receiver; we'd like to see how much this improves communications. The book he selected this year handles fundamentals of MIMO communications. MIMO is the latest and greatest achievement, he chose a book which goes from the basics. We might have single user MIMO or even multiple user MIMO: we're communicating to multiple phones through multiple antennas.

We can ask ourselves: what is the best that we can do? This is what our approach will be, so that we can know the boundaries. We'll follow the slides from the author of the book, hopefully they'll be good. We'll also have exercises with solutions: we can look at them from cambridge.org , specifically from the page of the book.

1.2 Primer

The first chapter is a primer in information theory and MSE estimation; we'll go fast as it should summarize stuff we know, as mutual information between two variables. !!: the signals are always complex-valued and the constellations are always normalized to have unit power. ∞-QAM: we increase the number of points making them denser inside a specific set.

1.2.1 Complex symmetric random variables, ∞-modulations

We'll start from chapter 1 of the book (000): *primer of information theory and MSE.*

We should already know this stuff, but we'll cover it in order to have a common starting ground.

∗Prof's backstory moment∗: In the end, he studies **physical layer** and that's it.

We ask ourselves: "**what is information**? When can we say that someone is giving us information?". ∗Professor begins ranting about information∗: information is related to **probability**! The more unlikely something is, the more informative it is!

Today we'll talk about **mutual information** between two random variables: the information a variable can supply to another variable.

We'll talk about **channel capacity**: the capacity at which a channel can convey information; it answers to "how much information can I send through this channel, reliably?". By reliably we mean that we can lower the probability of error as much as we want.

Binary coding and **decoding** are near optimum, but there can be other ways, which could be better: we'll show that binary is indeed **optimum**.

LMMSE is optimum when the signal and noise are **Gaussian**.

Let's start from **signals**; in the book, signals are normally **complex**, unless we specifically say otherwise. Normally **complex random variables** are **complexly symmetric**:

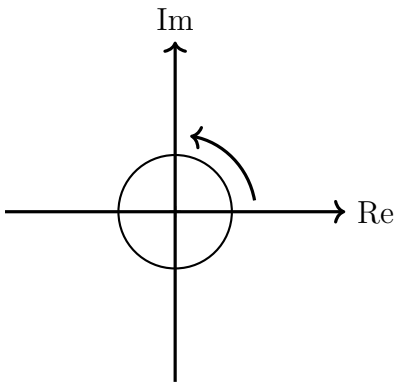
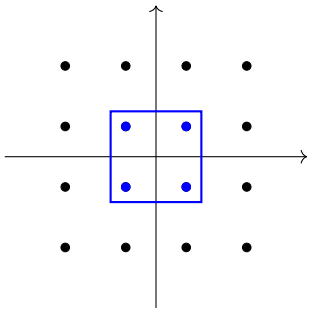


Figure 1.1: Rappresentazione del piano Im-Re con una PFD simmetrica centrale e freccia di rotazione.

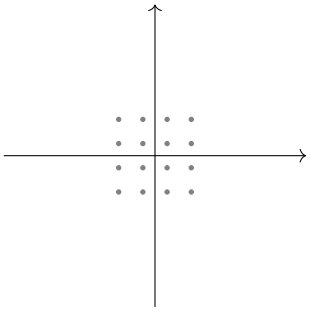
About **M-PSK** and **M-QAM**: M is usually a **power of 2**.

Let's see **16-QAM**:

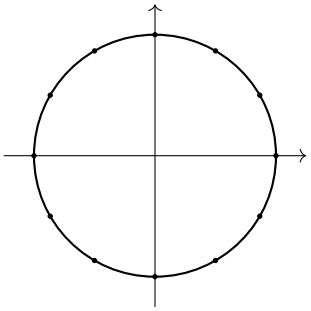


The power of the ones in blue is lower than the power of the other signals. In the book

they always keep the **same variance of the constellation**: the book squeezes the values so that the **power stays constant**.



The book also has **inf-PSK** and **inf-QAM**: We take the **normal constellation** and we keep **adding points**.



$\infty - \mathbf{PSK}$

It's a useful way to see what happens in the **asymptotic regime**. Also, the **PDF of a complex Gaussian variable exists** (see notes).

If σ^2 is the variance, the *Standard deviation* is $\sqrt{\sigma^2} = \sigma$

NOT $\pm\sigma^2$!

We'll try and be as precise as possible: there is no **negative standard deviation**! As an example:

$$f(x) = \sqrt{x} \in \mathbb{R} \text{ is def. only for } x \geq 0$$
$$x^2 = 16 \text{ has two sol.}$$

What about information? There have been many results, dating back to **Shannon**! (See pag 7 of the slides btw)

1.3 Discrete entropy, proper signals

1.3.1 Entropy

Entropy of a discrete-time random variable

$$\begin{aligned}\mathcal{H}(x) &= -\sum_x p_x(x)\log_2 p_x(x)\\ &= -\mathbb{E}[\log_2 p_x(x)].\end{aligned}$$

Joint entropy

$$\begin{aligned}\mathcal{H}(x_0,x_1) &= -\sum_{x_0}\sum_{x_1} p_{x_0x_1}(x_0,x_1)\log_2 p_{x_0x_1}(x_0,x_1)\\ &= -\mathbb{E}[\log_2 p_{x_0x_1}(x_0,x_1)].\end{aligned}$$

Nonnegative function that quantifies the amount of uncertainty in discrete valued RV x

Conditional entropy

$$\mathcal{H}(x|y) = -\sum_x\sum_y p_{xy}(x,y)\log_2 p_{x|y}(x|y).$$

Entropy connections

$$\mathcal{H}(x,y) = \mathcal{H}(x) + \mathcal{H}(y|x).$$

The entropy is used in statistical physics.

Joint entropy:

$$H(x_0,x_1) = \sum_{x_0}\sum_{x_1} p_{x_0,x_1}(x_0,x_1)\log_{1/2} p_{x_0,x_1}(x_0,x_1)$$

△ Diff. inform. <u>can</u> be negative

Conditional entropy:

$$H(x,y) = \sum_x\sum_y p_{x,y}(x,y)\log_2(p_{x|y}(x|y))$$

From this, we can use the Bayes theorem to derive:

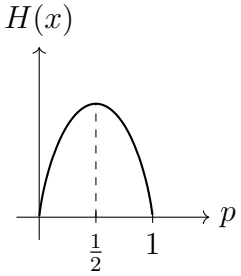
$$\begin{aligned}H(x,y) &= H(x|y) + H(y)\\ &= H(y|x) + H(x)\end{aligned}$$

Example: We have a Bernoulli variable

$$X = \{0, 1\} \qquad P = \{p, 1 - p\}$$

$H(X)$ will be ≤ 1, where the equality holds if $p = 1/2$.

If we plot it, we get:

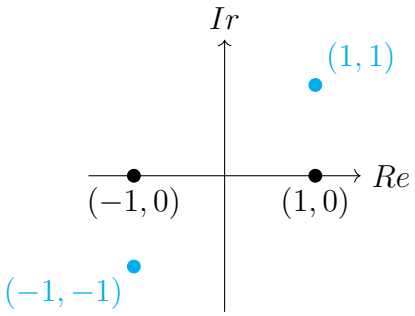


We get maximum information when we flip a ***fair*** coin!
If the coin always flips heads or tails, we get no information!

We could say that the messier the information, the most we get out of it.

Prof forgot stuff about signals: we're always talking about "proper" signals; the real and imaginary part of those numers are uncorrelated!

Properness is preserved by affine transformations. An intuitive reason why we consider only proper complex numbers is that if we have real and imaginary parts of a random variable:



$(-1, -1)$ **This is a fake complex variable; we want to avoid these cases.**

To elaborate further:

Proper complex

A random variable (or a random process/random vector) is said to be proper complex if it satisfies **Properness**:

Properness

A complex random variable is said to be **proper** if

$$E[x^2] = E[x]^2$$

holds. Properness requires $\mathrm{Re}[x], \mathrm{Im}[x]$ to be uncorrelated and have the same variance.

A complex random vector is said to be **proper** if

$$E[\mathbf{x}\mathbf{x}^T] = E[\mathbf{x}]E[\mathbf{x}]^T$$

holds, which is equal to asking that $\mathrm{Re}[\boldsymbol{x}], \mathrm{Im}[\boldsymbol{x}]$ have identical Covariance matrices and cross-covariance equal to 0.

Properness is preserved in affine transformations!

1.3.2 Entropy rate

Let's say we have a sequence of discrete random variables indexed by time, $x_0, \ldots, x_{N-1}$. If they are **IID**, then the entropy of the whole process grows linearly with N at a rate $\mathcal{H}(x_0)$.

In general, a process grows according to the so-called *entropy rate*

$$\mathcal{H} = \lim_{N \to +\infty} \frac{1}{N} \mathcal{H}(x_0, \ldots, x_{N-1}). \tag{1.32}$$

If the process is **stationary**, we can show that the entropy rate is equal to

$$\mathcal{H} = \lim_{N \to +\infty} \mathcal{H}(x_N | x_0, \ldots, x_{N-1}). \tag{1.33}$$

All of this can be extended for continuous processes to differential entropy.

A process is said to be **nonregular** if its present value is perfectly predictable from noiseless observations of the entire past, while the process is **regular** (which happens if $\mathfrak{h} > -\infty$) if its present value cannot be perfectly predicted from noiseless observations of the entire past.

1.3.3 Differential entropy

Differential entropy:

$$h(x) = \int f_x(x) \log_{1/2} f_x(x)\, dx = \mathbb{E}\left[\log_{1/2} f_x(x)\right]$$

For vectors:

$$h(\underline{x}) = \mathbb{E}\left[\log_{1/2} f_{\underline{x}}(\underline{x})\right]$$

The differential entropy rate does not measure the information contained in a random variable. If we only have a random variable, we only have a shot. In order to consider a **rate**, we consider a **random process**: in this case, we have a random variable every certain amount of time: we can say that we have a certain amount of information per second!

A binary random process with $p = 0.5$ which gives a value every second, gives 1 bit/s.

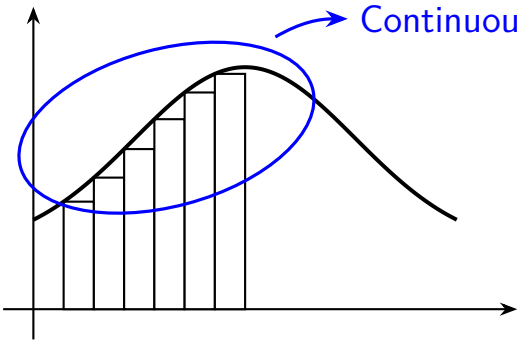
So the rate does not measure the information contained in a random variable. We also need to keep in mind that the differential entropy can be negative.

It's easy to prove that a **Gaussian distribution maximizes the differential entropy**, for a fixed variance.

The differential entropy of a b bit quantization of a random variable x is $h(x) + b$ (what happens if we quantize more and more finely?)

If we discretize a variable in b bits, then what we obtain is another random variable. What if we want to compute the differential entropy of that random variable?

Why do we use differential entropy even though we discretized? We just elongate it:



If we push b to inf, it diverges!
Btw if we want a reference for information theory, we can use the book "Cover & Thomas" (this is at page 247).

Pag 10 of the slides: how to calculate the differential entropy of a Gaussian complex vector.

Example 1.6 (Differential entropy of a complex Gaussian vector)

Let $\mathbf{x} \sim \mathcal{N}_{\mathcal{C}}(\boldsymbol{\mu}, \boldsymbol{R})$. From (C.15) and (1.22).

$$
\begin{aligned}
h(\mathbf{x}) &= -\mathbb{E}[\log_2 f_{\mathbf{x}}(\mathbf{x})] && (1.23)\\
&= \log_2 \det(\pi \boldsymbol{R}) + \mathbb{E}[(\mathbf{x}-\boldsymbol{\mu})^* \boldsymbol{R}^{-1}(\mathbf{x}-\boldsymbol{\mu})] \log_2 e && (1.24)\\
&= \log_2 \det(\pi \boldsymbol{R}) + \operatorname{tr}(\mathbb{E}[(\mathbf{x}-\boldsymbol{\mu})^* \boldsymbol{R}^{-1}(\mathbf{x}-\boldsymbol{\mu})]) \log_2 e && (1.25)\\
&= \log_2 \det(\pi \boldsymbol{R}) + \operatorname{tr}(\mathbb{E}[\boldsymbol{R}^{-1}(\mathbf{x}-\boldsymbol{\mu})(\mathbf{x}-\boldsymbol{\mu})^*]) \log_2 e && (1.26)\\
&= \log_2 \det(\pi \boldsymbol{R}) + \operatorname{tr}(\boldsymbol{R}^{-1} \underbrace{\mathbb{E}[(\mathbf{x}-\boldsymbol{\mu})(\mathbf{x}-\boldsymbol{\mu})^*]}_{\boldsymbol{R}}) \log_2 e && (1.27)\\
&= \log_2 \det(\pi \boldsymbol{R}) + \operatorname{tr}(\boldsymbol{I}) \log_2 e && (1.28)\\
&= \log_2 \det(\pi e \boldsymbol{R}), && (1.29)
\end{aligned}
$$

where in (1.25) we used the fact that a scalar equals its trace, while in (1.26) we invoked the commutative property in (B.26).

Note: $\det(\exp(\mathbf{A})) = \exp(\mathrm{tr}(\mathbf{A}))$

Example 1.5 (Differential entropy of a complex Gaussian scalar)

Let $x \sim \mathcal{N}_{\mathcal{C}}(\mu, \sigma^2)$. Invoking the PDF in (C.14),

$$h(x) = \mathbb{E}\left[\frac{|x-\mu|^2}{\sigma^2}\log_2 e + \log_2(\pi\sigma^2)\right] = \log_2(\pi e\sigma^2).$$

(Scalar result)

Equivalent scalar result

Professor freaks out over the fact that we don't know what the trace function is. The trace of a matrix is the sum of the diagonals. If we want to compute the sum of the eigenvalues, we can just compute the trace of a matrix.

btw $\det(\exp(\underline{A})) = \exp(\mathrm{tr}(\underline{A}))$

So we have that

$$h(\underline{x}) = \log_2(\det(\pi e\underline{R}))$$
$$h(x) = \log_2(\pi e\sigma^2)$$

At least not for a Gaussian R.V.!

The differential entropy DOES NOT depend on the mean of the variable!

Another very important thing is that the differential entropy is a property of the distribution of a random variable, not of the random variable itself. Two random variables having the same distribution will have the same random variable.

$$R.V. \text{ is a map } \quad \Omega \longrightarrow \mathcal{S}$$
$$\downarrow \qquad\qquad \downarrow$$
$$\textit{events} \qquad \textit{probability}$$

Elaborating a bit on the computation of the differential entropy: we have that

$$\begin{aligned} h(\boldsymbol{x}) &= -\mathbb{E}[\log_2 f_{\boldsymbol{X}}(\boldsymbol{x})] \qquad (1.23)\\ &= -\mathbb{E}\left[\log_2\left(\frac{1}{\det(\pi\boldsymbol{R}_x)}\exp(-(\boldsymbol{X}-\boldsymbol{\mu}_x)^*\boldsymbol{R}_x^{-1}(\boldsymbol{X}-\boldsymbol{\mu}_x))\right)\right]\\ &= \log_2(\det(\pi\boldsymbol{R}_x)) - \mathbb{E}\left[\frac{\ln(\exp(-(\boldsymbol{X}-\boldsymbol{\mu}_x)^*\boldsymbol{R}_x^{-1}(\boldsymbol{X}-\boldsymbol{\mu}_x)))}{\ln 2}\right]\\ &= \log_2(\det(\pi\boldsymbol{R}_x)) - \mathbb{E}\left[\frac{-(\boldsymbol{X}-\boldsymbol{\mu}_x)^*\boldsymbol{R}_x^{-1}(\boldsymbol{X}-\boldsymbol{\mu}_x)}{\ln 2}\right]\\ &= \log_2(\det(\pi\boldsymbol{R}_x)) + \mathbb{E}\left[(\boldsymbol{X}-\boldsymbol{\mu}_x)^*\boldsymbol{R}_x^{-1}(\boldsymbol{X}-\boldsymbol{\mu}_x)\right]\log_2 e \qquad (1.24)\end{aligned}$$

and now we observe that $(\boldsymbol{X}-\boldsymbol{\mu}_x)^*\boldsymbol{R}_x^{-1}(\boldsymbol{X}-\boldsymbol{\mu}_x)$ actually is a scalar (given that $\boldsymbol{x}$ is a $N\times 1$ vector, we're multiplying $(1\times N)\cdot(N\times N)\cdot(N\times 1)\to 1$): we can then exploit the fact that a scalar is equal to its trace together with the fact that the trace is a scalar operation to get

$$
\begin{aligned}
&= \log_2(\det(\pi\boldsymbol{R}_x)) + \operatorname{Tr}\left[\mathbb{E}\left[(\boldsymbol{X}-\boldsymbol{\mu}_x)^*\boldsymbol{R}_x^{-1}(\boldsymbol{X}-\boldsymbol{\mu}_x)\right]\right]\log_2 e && (1.25)\\
&= \log_2(\det(\pi\boldsymbol{R}_x)) + \mathbb{E}\left[\operatorname{Tr}\left[(\boldsymbol{X}-\boldsymbol{\mu}_x)^*\boldsymbol{R}_x^{-1}(\boldsymbol{X}-\boldsymbol{\mu}_x)\right]\right]\log_2 e && \text{Linearity}\\
&= \log_2(\det(\pi\boldsymbol{R}_x)) + \mathbb{E}\left[\operatorname{Tr}\left[\boldsymbol{R}_x^{-1}(\boldsymbol{X}-\boldsymbol{\mu}_x)(\boldsymbol{X}-\boldsymbol{\mu}_x)^*\right]\right]\log_2 e && \text{Ciclicity}\\
&= \log_2(\det(\pi\boldsymbol{R}_x)) + \operatorname{Tr}\left[\boldsymbol{R}_x^{-1}\boldsymbol{R}_x\right]\log_2 e\\
&= \log_2(\det(\pi\boldsymbol{R}_x)) + \operatorname{Tr}\left[\boldsymbol{I}\right]\log_2 e && (1.28)
\end{aligned}
$$

if then we consider $\boldsymbol{R}_x, \boldsymbol{I}$ as $N\times N$ matrices (which they are, this is just needed to say that N is their measure), we'll have

$$
\begin{aligned}
&= \log_2(\det(\pi\boldsymbol{R}_x)) + N\log_2 e\\
&= \log_2(e^N\det(\pi\boldsymbol{R}_x))
\end{aligned}
$$

and, finally, for a matrix $\boldsymbol{A}$ of dimensionality $N\times N$ it holds that

$$a^N\det(\boldsymbol{A}) = \det(a\boldsymbol{A}),$$

so in our situation this is

$$= \log_2(\det(\pi e\boldsymbol{R}_x)). \tag{1.29}$$

Whew, that was a lot!

Finally, we can simplify this for the scalar case as the determinant of a scalar is the scalar itself, getting that

$$h(x) = \log_2(\pi e\sigma^2).$$

> Btw the B.26 relation that they mentioned is
>
> $$\operatorname{tr}(\boldsymbol{AB}) = \operatorname{tr}(\boldsymbol{BA}), \tag{B.26}$$
>
> which was here applied to $\boldsymbol{A} = (\boldsymbol{x}-\boldsymbol{\mu})^*\boldsymbol{R}^{-1}, \boldsymbol{B} = (\boldsymbol{x}-\boldsymbol{\mu})$.

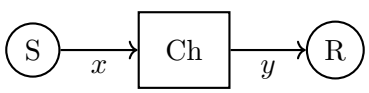
1.4 Information divergence, mutual information

We have that the Kullback-Leibler divergence is defined as

$$\mathcal{D}(p||q) = \begin{cases}\sum_x p(x)\log_2\left(\frac{p(x)}{q(x)}\right) & \text{discrete } x\\ \int p(x)\log_2\left(\frac{p(x)}{q(x)}\right) & \text{continuous } x\end{cases} \tag{1.34+1.36}$$

this is also called the *relative entropy*.

Another way to measure such thing is the Earth-Mover distance.



The mutual information quantifies the reduction in uncertainty between two random variables if one of them is observed.

Discrete:

$$\mathrm{I}(s;y) = H(s) - H(s|y) = H(y) - H(y|s)$$

Between s and y

$$\begin{aligned} &= D(p_{sy}||p_s p_y) \\ &= \sum_s \sum_y p_{sy}(s,y) \log_2 \left(\frac{p_{sy}(s,y)}{p_s(s) p_y(y)} \right) \end{aligned}$$

Cont:

$$\begin{aligned} &= \mathbb{E}[\text{information density}] \equiv \mathbb{E}[i(s;y)] \\ \mathrm{I}(s;y) &= h(s) - h(s|y) = h(y) - h(y|s) \\ &= D(f_{sy}||f_s f_y) \\ &= \iint f_{sy}(s,y) \log_2 \left(\frac{f_{sy}(s,y)}{f_s(s) f_y(y)} \right) ds\, dy \end{aligned}$$

He talked about the Landau symbols; he didn't say anything relevant really.

1.4.1 Power series expansions

Useful means to approximate functions especially low or high SNR.

- **Zero order expansion about 0**

$$\boxed{f(x) = f(0) + \mathcal{O}(x)}$$

- **First order expansion about 0**

$$\boxed{f(x) = f(0) + f'(0)x + \mathcal{O}(x^2)}$$

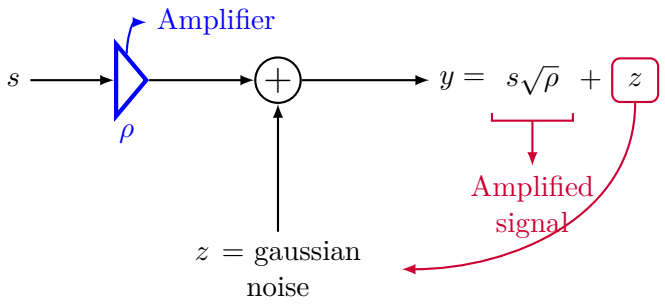
- **First order expansion about Infinity**

$$\boxed{f(1/x) = f(0) + \frac{f'(0)}{x} + \mathcal{O}\left(\frac{1}{x^2}\right)}$$

Mathematica snippet:

```
In[11]:= Series[f[x], {x, 0, 2}]
Out[11]= f[0] + f'[0]x + 1/2 f''[0]x^2 + O[x]^3
```

Important! → SL. 16



We can ask ourselves: how much is the gaussian noise corrupting what I'm sending?

We amplify ρ (which is $\sqrt{\rho}$ in power) $\Rightarrow$ we get an ampl. of $\sqrt{\rho}$ on power!

1.4.2 Mutual information of a Gaussian scalar through an AWGN channel

We were calculating the **mutual information** for a Gaussian complex scalar.

We talked about the relation $y = \sqrt{\rho}s + z$. We have that s, z are **independent**. It might seem weird that we're putting noise in a channel, but it will make sense later.

Our constellation input always will have $\sigma^2 = 1$, such that $s \sim \mathcal{N}_{\mathcal{C}}(0, 1)$; we'll also have a noise $z \sim \mathcal{N}_{\mathcal{C}}(0, 1)$. We have that y is made of a sum of Gaussian independent random variables, and will thus be a Gaussian random variable, with a variance given by a sum of variances: we'll have

- $y \sim \mathcal{N}_{\mathcal{C}}(0, 1 + \rho)$
- $y|s \sim \mathcal{N}_{\mathcal{C}}(\sqrt{\rho}s, 1)$: this is y with s being fixed; we fix the $\sqrt{\rho}s$ in y's equation. We now want to calculate the mutual information between the input and the output of the signal.

We know the differential entropy of a Gaussian scalar and we know that

$$\begin{aligned}\mathcal{I}(\rho) &= I(s; \sqrt{\rho}s + z) \\ &= h(\sqrt{\rho}s + z) - h(\sqrt{\rho}s + z|s) \\ &= h(\sqrt{\rho}s + z) - h(z) \\ &= \log_2(\pi e(1 + \rho)) - \log_2(\pi e) \\ &= \log_2(1 + \rho). \end{aligned} \tag{1.51}$$

Relevant comments:

- In $h(\sqrt{\rho}s + z|s) = h(z)$, we know that we've already **fixed** s; s multiplied by whichever scalar won't bring any extra information than z, as we already know what s is; for this reason, the differential entropies are the same.

If we then consider $\rho \to 0$, we have

$$\log_e(1+x) \xrightarrow{x\to 0} x - \tfrac{1}{2}x^2 + \mathcal{O}(x^3)$$

So we need to **convert to base** e and we'll then be able to use the expansion:

$$\log_2(1+\rho) = \left(\rho - \tfrac{1}{2}\rho^2\right)\log_2 e + \mathcal{O}(\rho^3)$$

And **for large** ρ we'll then have

$$\log_2(1+\rho) \xrightarrow{\rho\to\infty} \log_2(\rho) + \mathcal{O}(1/\rho)$$

Note

This is **crucially important** as it allows to model an AWGN channel! It allows us to define the **capacity** as

$$\frac{1}{2}\log(1+\mathrm{SNR})$$

(we don't have 1/2 since this is complex; we'd have 1/2 if it wasn't complex!).

Chapter 2

Chapter 2 Week 27 October - 2 November

2.1 MIMO Intution

We'll use random vectors in order to model MIMO communications Our book is **MIMO**! For a long time we had a setup with a transmitter and a receiver.

People asked: what can we do in order to increase the **spectral efficiency** of our signals? The spectral efficiency is how many **bit/s/Hz** we can squeeze.

The spectral efficiency we've reached is between **1 and 5 bit/s/Hz**; we generally have **4 bit/s/Hz**.
$B \cdot \nu$ is how many bits we can send (B = bandwidth, ν = spectral efficiency).

What if we have **two antennas** at the transmitter and **two at the receiver**? It's been shown that by doing such, **it's like having twice the bandwidth**: this is the whole concept of **MIMO**.
Of course we have to pay **double**.
With two antennas we're transmitting two symbols, aka a **vector**:

$$\left[\begin{array}{c} T_x \\ \hline S_1 \\ S_2 \end{array}\right]$$

And the received signal vector is:

$$\left[\begin{array}{c} L_{RC} \\ \hline y_1 \\ y_2 \end{array}\right] = A \left[\begin{array}{c} S_1 \\ S_2 \end{array}\right] + \left[\begin{array}{c} z_1 \\ z_2 \end{array}\right]$$

We thus need to use **vectors**, we can't stay with **scalars**!

2.2 Mutual information of a gaussian vector through an AWGN channel (+figure of noise)

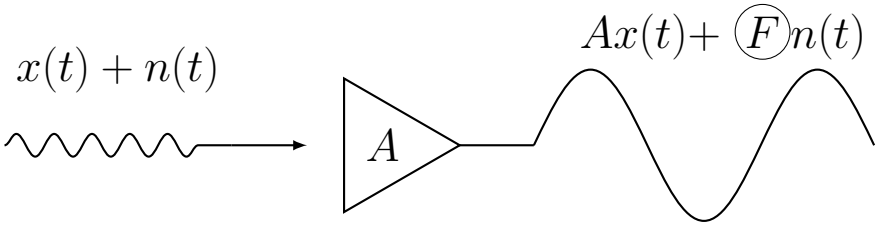
Let's say we have a vector **s** and a matrix **A** (which is needed since every signal is communicating with each antenna!): **we're now in the MIMO setting**.
We have a **Gaussian vector s**, a **matrix A** and a **noise Gaussian vector z**; we'll have an output

$$\mathbf{y} = \sqrt{P_A}\mathbf{s} + \mathbf{z},$$

where **A** is considered **deterministic** (we'll see it's not really true), **z** is the **noise**, which originates from **thermal noise**: **electrons are not still**; the more we increase the temperature, the more **electrons move**; we want to transmit a certain voltage, but we'll measure a certain fluctuating value added to our transmitted signal: the **fluctuating addition comes from thermal noise**!

2.2.1 Noise figure

Noise figure
What about **noise figure**? When we amplify a signal, do we pay a price? We're amplifying the noise as well, but we might amplify the noise more: the amplifier will add its own noise to the noise, which is F, the noise figure:



We now want to calculate the mutual information between source and output; the idea is the same, only with matrices and vectors; we have

$$\begin{aligned}
\mathcal{I}(\rho) &= I(\boldsymbol{s}; \sqrt{\rho}\boldsymbol{A}\boldsymbol{s} + \boldsymbol{z}) && (1.74)\\
&= h(\sqrt{\rho}\boldsymbol{A}\boldsymbol{s} + \boldsymbol{z}) - h(\sqrt{\rho}\boldsymbol{A}\boldsymbol{s} + \boldsymbol{z}|\boldsymbol{s}) && (1.75)\\
&= h(\sqrt{\rho}\boldsymbol{A}\boldsymbol{s} + \boldsymbol{z}) - h(\boldsymbol{z}) && (1.76)\\
&= \log_2 \det(\pi e(\rho \boldsymbol{A}\boldsymbol{R}_s\boldsymbol{A}^* + \boldsymbol{R}_z)) - \log_2 \det(\pi e \boldsymbol{R}_z) && (1.77)\\
&= \log_2 \det(\boldsymbol{I} + \rho \boldsymbol{A}\boldsymbol{R}_s\boldsymbol{A}^*\boldsymbol{R}_z^{-1}) \ , && (1.78)
\end{aligned}$$

with (1.77) following from the definition of differential entropy of a gaussian vector. The correlation matrix of $\sqrt{\rho}\boldsymbol{A}\boldsymbol{s}$ is $\rho\boldsymbol{A}\boldsymbol{R}_s\boldsymbol{A}^*$; we then have a sum, so we'll have $\boldsymbol{R}_z$. The fact that in the last passage (1.78) we have $\boldsymbol{R}_z$ at the denominator, means we'll be multiplying by $\boldsymbol{R}_z^{-1}$; this follows from the fact that

$$\frac{\det(\boldsymbol{A})}{\det(\boldsymbol{B})} = \det(\boldsymbol{A}\boldsymbol{B}^{-1}) \,,$$

here applied with $\boldsymbol{A} = \rho\boldsymbol{A}\boldsymbol{R}_s\boldsymbol{A}^* + \boldsymbol{R}_z$ and $\boldsymbol{B} = \boldsymbol{R}_z$.

How do we know that $\sqrt{\rho}\boldsymbol{A}\boldsymbol{s} + \boldsymbol{z}$ is Gaussian?

I think the idea is that we can consider $\sqrt{\rho}\boldsymbol{A}\boldsymbol{s} + \boldsymbol{z}$ as a multiplication between the matrix

$$[\sqrt{\rho}\boldsymbol{A} \quad 1]$$

and the vector

$$\begin{bmatrix} \boldsymbol{s} \\ \boldsymbol{z} \end{bmatrix},$$

which will result in a random vector with distribution

$$\sim \mathcal{N}_{\mathbb{C}} \left([\sqrt{\rho}\boldsymbol{A} \quad 1] \cdot \begin{bmatrix} 0 \\ 0 \end{bmatrix}, [\sqrt{\rho}\boldsymbol{A} \quad 1] \cdot \begin{bmatrix} \boldsymbol{R}_s & 0 \\ 0 & \boldsymbol{R}_z \end{bmatrix} \cdot \begin{bmatrix} \sqrt{\rho}\boldsymbol{A}^* \\ 1 \end{bmatrix} \right),$$

according to

Transformation of a Gaussian vector

If we have:

- A Gaussian vector $\boldsymbol{x} \sim \mathcal{N}_{\mathbb{C}}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ of dimensions $N \times 1$,
- a matrix $\boldsymbol{A}$ of dimensions $M \times N$
- a vector $\boldsymbol{b}$ of dimensions $M \times 1$

we have that the vector

$$\boldsymbol{y} = \boldsymbol{A}\boldsymbol{x} + \boldsymbol{b}$$

will be distributed according to

$$\boldsymbol{y} \sim \mathcal{N}_{\mathbb{C}}(\boldsymbol{A}\boldsymbol{\mu} + \boldsymbol{b}, \ \boldsymbol{A}\boldsymbol{\Sigma}\boldsymbol{A}^*).$$

▶ Extension of Uni generale.

Derivative of logarithm of determinant

In order to go on, we consider the following relations:

$$\left.\frac{\partial}{\partial\rho} \log_e \det(\boldsymbol{I} + \rho\boldsymbol{B})\right|_{\rho=0} = \mathrm{tr}(\boldsymbol{B})$$
$$\left.\frac{\partial^2}{\partial\rho^2} \log_e \det(\boldsymbol{I} + \rho\boldsymbol{B})\right|_{\rho=0} = -\mathrm{tr}(\boldsymbol{B}^2)$$

We can then apply some linear algebra: we'll get that for $\rho \to 0$ follows

$$\mathcal{I}(\rho) = \left[\operatorname{tr}(\boldsymbol{A}\boldsymbol{R}_{\boldsymbol{s}}\boldsymbol{A}^*\boldsymbol{R}_{\boldsymbol{z}}^{-1})\rho - \frac{1}{2}\operatorname{tr}((\boldsymbol{A}\boldsymbol{R}_{\boldsymbol{s}}\boldsymbol{A}^*\boldsymbol{R}_{\boldsymbol{z}}^{-1})^2)\rho^2\right]\log_2 e + o(\rho^2)$$

About this passage: it all depends on the fact that log det is a **convex function**! We know the derivative of log det and we're applying it to the fact that we're summing the eigenvalues: that's how we get the **trace**!

Page **74** of a linear algebra book he recommended

This is a very useful result!

If $N_{\boldsymbol{s}}, N_{\boldsymbol{y}}$ are the dimensions of the vectors, for $\rho \to +\infty$ we get the result

$$\mathcal{I}(\rho) = \min(N_{\boldsymbol{s}}, N_{\boldsymbol{y}})\log_2 \rho + \log_2 \det(\boldsymbol{A}\boldsymbol{R}_{\boldsymbol{s}}\boldsymbol{A}^*\boldsymbol{R}_{\boldsymbol{z}}^{-1}) + O\left(\frac{1}{\rho}\right)$$

Professor said that this equation has a minor detail which is **wrong**, proceeded into not telling us what it was :D

2.3 Gaussian mutual information for an arbitrary constellation

Let's see the Gaussian mutual information for an arbitrary constellation.

Let $\boldsymbol{s}$ be a zero-mean unit-variance discrete random variable taking values in $\boldsymbol{s}_0, \dots, \boldsymbol{s}_{M-1}$ with probabilities $p_0, \dots, p_{M-1}$. This subsumes M-**PSK**, M-**QAM**, and any other discrete constellation. The PDF of $\boldsymbol{y} = \sqrt{\rho}\boldsymbol{s} + \boldsymbol{z}$ equals

$$f_{\boldsymbol{y}}(y) = \frac{1}{\sqrt{2\pi}}\sum_{m=0}^{M-1} p_m e^{-|y-\sqrt{\rho}\boldsymbol{s}_m|^2} \tag{1.65v}$$

whereas $y|\boldsymbol{s} \sim \mathcal{N}_{\mathbb{C}}(\sqrt{\rho}\boldsymbol{s}, 1)$. Thus,

$$\mathcal{I}^{\text{aw}}(\rho) = I(\boldsymbol{s}; \sqrt{\rho}\boldsymbol{s} + \boldsymbol{z}) \tag{1.66}$$

$$= -\int f_{\boldsymbol{y}}(y)\log_2 f_{\boldsymbol{y}}(y)\,\mathrm{d}y - \log_2(\pi e) \tag{1.67}$$

with integration over the complex plane.

$$\mathcal{I}^{\text{aw}}_{\text{aw}}(\rho) = \log_2 M - \epsilon \qquad \textbf{Large a first order expansion} \qquad \log_2 e = \frac{d^2_{\min}}{4}\rho + o(\rho)$$

dove

$$d_{\min} = \min_{k \neq l} |\boldsymbol{s}_k - \boldsymbol{s}_l|$$

Comments:

- In order to obtain (1.65) we used

$$f_{\boldsymbol{y}}(y) = \sum_{m=0}^{M-1} f_{\boldsymbol{y}|\boldsymbol{s}}(y|\boldsymbol{s}_m) p_{\boldsymbol{s}}(\boldsymbol{s}_m),$$

  with $p_{\boldsymbol{s}}(\boldsymbol{s}_m) = p_m$.
- (1.67) is just $h(\boldsymbol{y}) - h(\boldsymbol{y}|\boldsymbol{s})$, considering that $\boldsymbol{y}|\boldsymbol{s}$ is a **Gaussian RV with variance 1** and thus $h(\boldsymbol{y}|\boldsymbol{s}) = h(\boldsymbol{z}) = \log_2(\pi e)$.
- We don't see σ^2 since it's $\sigma_{\boldsymbol{z}}^2 = 1$!
- ϵ is first defined here

(1.66) is the definition of mutual information. Solving the integral is pretty difficult.

What professor wants us to remember though is that

$$\mathcal{I}^{\text{aw-ary}}(\rho) \xrightarrow{\rho \to +\infty} \log_2(M) - \epsilon,$$

with

$$\log_2(\epsilon) = \frac{d_{\min}^2}{4}\rho + o(\rho)$$

intuitively, a constellation with M points asymptotically gives $\log_2(M)$ bits of information per channel use; of course this is not the case and we need to lower that term using ϵ, which brings us from the **best case scenario** to a more realistic case.

2.4 Mutual information of an encoded word

2.4.1 Theoretical treatment: results for generic $\boldsymbol{s}, \tilde{\boldsymbol{y}}$ (+for iid values and for a stationary channel)

We have an abstraction of the communication link:

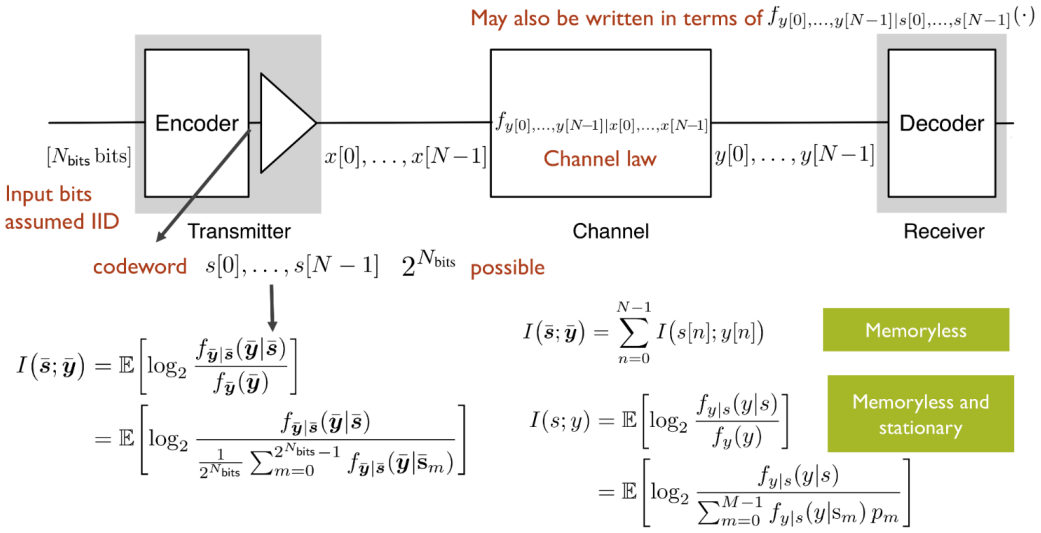


Figure 2.1: Abstraction of the communication link

We have an encoder, which maps a signal to a codebook with $2^{N_{\text{bits}}}$ words. We send a codeword of N bits $x[n], n \in [0, N-1]$. We then send our signal through the channel, which is characterized by **noise**, **amplification**, ...; we'll receive a whole word $\boldsymbol{y}[n], n \in [0, N-1]$. If we want to calculate the **mutual information between input words and output words**, we need to use the **conditioned information function**, not the joint one: we know the probability of receiving a certain message $\boldsymbol{y} = [y[0], \dots, y[N-1]]$ (as it is $f_{\boldsymbol{y}}$) when sending a certain $\boldsymbol{s} = [s[0], \dots, s[N-1]]$ and we need to work with that!

If we assume that the various symbols are **equiprobable**, we find that

$$f_{\boldsymbol{y}}(\boldsymbol{Y}) = \sum_{m=0}^{2^{N_{\text{bits}}}-1} f_{\boldsymbol{y}|\boldsymbol{s}}(\boldsymbol{Y}|\boldsymbol{s}_m) p_{\boldsymbol{s}}(\boldsymbol{s}_m) \overset{\text{equiprobable } \boldsymbol{s}_m}{=} \sum_{m=0}^{2^{N_{\text{bits}}}-1} f_{\boldsymbol{y}|\boldsymbol{s}}(\boldsymbol{Y}|\boldsymbol{s}_m) \frac{1}{2^{N_{\text{bits}}}}$$

and this enables us to write

$$\begin{aligned} I(\boldsymbol{s};\boldsymbol{y}) &= \mathcal{H}(\boldsymbol{y}) - \mathcal{H}(\boldsymbol{y}|\boldsymbol{s}) && (1.88) \\ &= \mathbb{E}\left[\log_2\left(\frac{f_{\boldsymbol{y}|\boldsymbol{s}}(\boldsymbol{y}|\boldsymbol{s})}{f_{\boldsymbol{y}}(\boldsymbol{y})}\right)\right] \\ &= \mathbb{E}\left[\log_2\left(\frac{f_{\boldsymbol{y}|\boldsymbol{s}}(\boldsymbol{y}|\boldsymbol{s})}{\frac{1}{2^{N_{\text{bits}}}}\sum_{m=0}^{2^{N_{\text{bits}}}-1} f_{\boldsymbol{y}|\boldsymbol{s}}(\boldsymbol{Y}|\boldsymbol{s}_m)}\right)\right]. && (1.89) \end{aligned}$$

If the channel is **memory-less** (which might not be true: $y[1]$ might depend on $y[0]$ etc etc), the mutual information of input and output is the **sum of mutual information of every couple of input and output**: we don't have vectors anymore (and we like this!).

If this is the case, the channel law will factor out as

$$f_{\boldsymbol{y}|\boldsymbol{s}}(\boldsymbol{y}|\boldsymbol{s}) = \prod_{n=0}^{N-1} f_{y[n]|s[n]}(y[n]|s[n]).$$

with

$$I(s[n]; y[n]) = \mathbb{E}\left[\log_2 \frac{f_{y[n]|s[n]}(y[n]|s[n])}{f_{y[n]}(y[n])}\right]. \quad (1.93)$$

Hidden message
Looks like one hidden message is that if the conditioned probability factorizes, we can easily rewrite the mutual information as a summation. This follows from the fact that if the single elements are iid, then the expected value factorizes as well.

If the channel is also stationary, we can simplify further, as all the symbols will have the same statistics: $s \equiv s[n]\ \forall n$ and $y \equiv y[n]\ \forall n$ will hold, and can write (1.93) as

$$I(s;y) = \mathbb{E}\left[\log_2 \frac{f_{y|s}(y|s)}{f_y(y)}\right] \tag{1.94}$$

$$= \mathbb{E}\left[\log_2 \frac{f_{y|s}(y|s)}{\sum_{m=0}^{M-1} f_{y|s}(y|s_m)p_m}\right], \tag{1.95}$$

with p_m being the probability that a symbol s is s_m; we can of course further simplify this writing if we assume equiprobability, $p_m = 1/M$.

This comes from the fact that any output does not store any information about any input other than the one which corresponds to it:

$$I(s[k];y[n]) = \begin{cases} 0 & k \neq n \\ \text{Something} & \text{otherwise} \end{cases}.$$

2.5 Memoryless channel law with Gaussian noise

Let's see an example; if our output is defined as

$$y[n] = \sqrt{\rho}s[n] + z[n] \qquad n = 0, \dots, N-1 \tag{1.96}$$

and we have that $z[0], \dots, z[N-1]$ are IID with $z \sim \mathcal{N}_{\mathbb{C}}(0,1)$ the channel law will be the same as z, now centered in mean in the same value as the transmitted s:

$$f_{y|s}(y|s) = \frac{1}{\pi} e^{-|y-\sqrt{\rho}s|^2}. \tag{1.97}$$

The channel law is the probability that we receive y when we're transmitting s.

2.5.1 Capacity of a channel, information stability, spectral efficiency (+codeword error, information density)

We come to an extra important concept: **capacity**. We can express it in different ways; we'll see **bit/s**; when we'll put the symbols one after the other.

Ideally we must study the book, in order to fully grasp these concepts

The **codeword error** probability for equiprobable words of length N is

$$p_e = \frac{1}{2^N} \sum_{m=0}^{2^N-1} P[\hat{w} \neq m | w = m],$$

where w is the transmitted word and $\hat{w}$ is the received word.

Capacity (bits/symbol)
The capacity C in bits per symbol is the highest rate that can be achieved such that for large values of N we have

$$p_e \overset{N\to+\infty}{\to} 0.$$

The **information density** between an input word $\bar{\boldsymbol{s}}$ and an output word $\bar{\boldsymbol{y}}$ is defined as

$$i(\bar{\boldsymbol{s}};\bar{\boldsymbol{y}}) = \log_2\left(\frac{f_{\bar{\boldsymbol{s}},\bar{\boldsymbol{y}}}(\bar{\boldsymbol{S}},\bar{\boldsymbol{Y}})}{f_{\bar{\boldsymbol{s}}}(\bar{\boldsymbol{S}})f_{\bar{\boldsymbol{y}}}(\bar{\boldsymbol{Y}})}\right) .$$

If the **channel** is **information stable**, then we have that

$$\lim_{N\to+\infty}\frac{1}{N}i(\bar{\boldsymbol{s}},\bar{\boldsymbol{y}}) = \lim_{N\to+\infty}\frac{1}{N}\mathbb{E}[i(\bar{\boldsymbol{s}},\bar{\boldsymbol{y}})] \overset{*}{=} \lim_{N\to+\infty}\frac{1}{N}I(\bar{\boldsymbol{s}},\bar{\boldsymbol{y}}) .$$

Professor's definition of information stable
A channel is information stable if the mutual information does not vary if we jump in the future or past; it's not taking the first symbols and conveying less information than the symbols coming in later.

This relation is saying that the information conveyed by $y[0],\dots,y[N-1]$ about $s[0],\dots,s[N-1]$ is invariant, provided that N is large enough

$*$: note that, by definition,

$$\mathbb{E}[i(s;y)] = I(s;y) .$$

Sufficient condition for information stability
A sufficient condition for a channel to be information stable is that **the channel is stationary and ergodic** (ref)

Ergodicity in mean
We have a random process $x(n)$ and we know that $\mathbb{E}[x(n)] = B$.
A signal is **ergodic in mean** if

$$\lim_{N\to+\infty}\sum_{n=0}^{N-1}\frac{x(n)}{N} = B :$$

the average in time is the same as the average taken in the probability space

If the channel is stationary and ergodic, then

$$C = \max_{\text{signal constraints}}\left(\lim_{N\to+\infty}\frac{1}{N}I(\bar{\boldsymbol{s}},\bar{\boldsymbol{y}})\right) .$$

The maximization is over the joint distribution of the unit-power codeword symbols $s[0],\dots,s[N-1]$.

Zero mean signals
The mean has no influence over the mutual information (and thus the capacity), but it does influence the average power spent for the transmission; for this reason, from now onwards, we'll only consider zero-mean signals.

We want the codeword error probability to go to zero for long words. We define an **information stable channel**; we then have the information density: if it's information stable, the information density must become the mutual information for large Ns. The capacity is the **maximum mutual information which we can achieve by changing the distribution of the input signal**. We have very long codewords and we want the probability of error on the codeword to be as low as we want.

If we have a pipe, the capacity is how much water we can push through without losing any water

In order to reach the capacity, what we can do is changing the input distribution of the symbols.

If the channel is **static, memoryless and ergodic**, then we simply have

$$C = \max_{\text{signal constraints}} I(s; y).$$

The bitrate R at which we can reliably communicate with a period T satisfies $R \leq C/T$. From the sampling theorem we have $1/T \leq B$, where B is the passband bandwidth of our transmission. By joining the two conditions, we obtain that

$$\underbrace{\frac{R}{B}}_{\nu,\text{ spectral efficiency}} \leq C \leftrightarrow \nu \leq C,$$

where C uses the alternative measure bit/s/Hz and where the equality is reached asymptotically.

This is telling us that the capacity is not only the maximum bitrate, but also the maximum spectral capacity which we can reach.

The capacity involves searching in all the possible distributions, we'll always stay underneath it, and that's something we always need to take into account!

2.6 BICM architecture (actually: Information loss when assuming bit independence, interleavers, soft decoding)

From the book: In paragraphs **before 1.5.4**, the book showed that using **binary encoding** and **constellation mapping** together yields no loss of optimality. We can ask ourselves whether this also holds when we combine **soft demapping** and **binary decoding**. In order to investigate this, we consider a stationary and memoryless channel as well as an M-point equiprobable constellation.

A primer on information theory and MMSE estimation (pp. 3-56), p. 28
In this setting, codewords with IID entries are optimum and thus bits mapped to distinct symbols can be taken to be independent. However, **the channel does introduce dependencies among same-symbol bits**. Even with the coded bits produced by the binary encoder being statistically independent (as we know from *past paragraphs*), after demapping at the receiver dependencies do exist among the soft values for bits that traveled on the same symbol. Unaware, a binary decoder designed for IID bits ignores these dependencies and regards the channel as being memoryless, not only at a symbol level but further at a bit level [91]. Let us see **how much information can be recovered under this premise**.

First of all, we look at the decoder counterpart of (1.94), (1.95), if the channel is also stationary, we can simplify further, as all the symbols will have the same statistics: $s \equiv s[n]\ \forall n$ and $y \equiv y[n]\ \forall n$ will hold, and can write (1.93) as

$$I(s;y) = \mathbb{E}\left[\log_2 \frac{f_{y|s}(y|s)}{f_y(y)}\right] \tag{1.94}$$

$$= \mathbb{E}\left[\log_2 \frac{f_{y|s}(y|s)}{\sum_{m=0}^{M-1} f_{y|s}(y|s_m)p_m}\right] \tag{1.95}$$

with p_m being the probability that a symbol s is s_m; we can of course further simplify this writing if we assume equiprobability, $p_m = 1/M.$, which is

$$I_{\text{aw}}^{\text{BICM}} = \sum_{k=0}^{m_k-1} I(\boldsymbol{b}_k;\boldsymbol{y})$$

after much hard work, we find that it is

$$I_{\text{aw}}^{\text{BICM}} = \mathbb{E}_{\boldsymbol{y}}\left[\sum_{k=1}^{m_k} \mathbb{E}_{\boldsymbol{s}}\left[\log_2 \frac{\sum_{i=0}^{2^{m_k-1}} \mathbb{E}_{\boldsymbol{s}_{\setminus k}}[f_{\boldsymbol{y}|\boldsymbol{s}}(\boldsymbol{y}|\boldsymbol{s}_i)]}{\sum_{i=0}^{2^{m_k-1}} f_{\boldsymbol{y}|\boldsymbol{s}}(\boldsymbol{y}|\boldsymbol{s}_i)}\right]\right] \tag{1.137}$$

where $\boldsymbol{s}[k]$ are the subsets of constellation points where k-th bit is either 0 or 1; just to give an example:

With constellations such as BPSK and QPSK, where a single coded bit is mapped to the in-phase and quadrature dimensions of the constellation and thus **no dependencies among same-symbol coded bits**, we have that $\sum_k I(b_k;y) = I(\boldsymbol{s};\boldsymbol{y})$, since the binary decoder is not disregarding information. Instead, **if the decoder disregards information** (acquiring information about other bits in the same symbol), the binary coding yields $\sum_k I(b_k;y) < I(\boldsymbol{s};\boldsymbol{y})$.

> **Skipped**
> Several examples have been skipped: 1.20, 1.21, 1.22, 1.23

Anyhow, it's true that the difference between $\sum_k I(b_k;y)$ and $I(\boldsymbol{s};\boldsymbol{y})$ is generally tiny, if the mapping of coded bits to constellation points is chosen wisely. An example of wise choice is **Gray mapping**, where neighboring constellation points only differ of one bit.

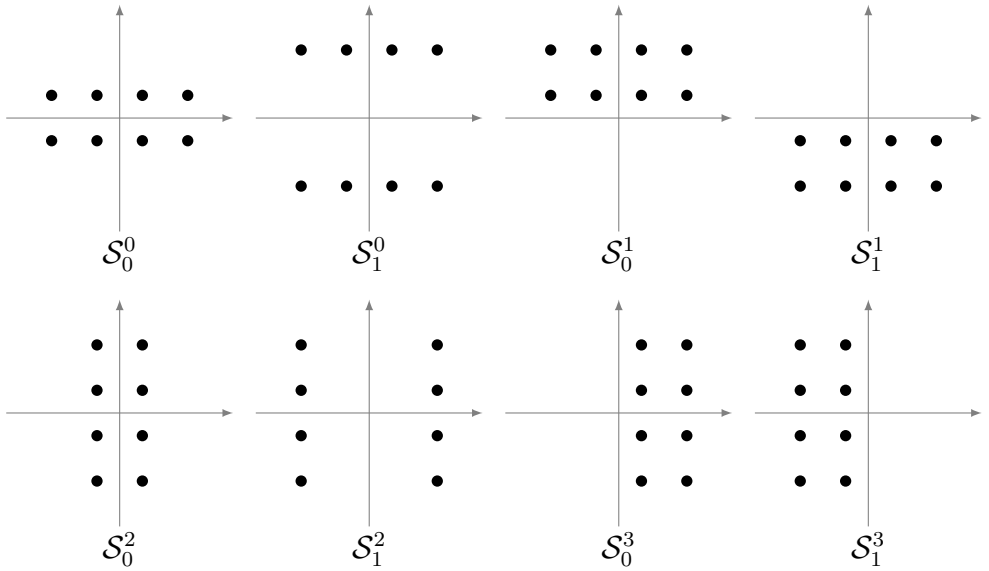


Figure 2.2: Above, 16-QAM constellation with Gray mapping. Below, subsets $\mathcal{S}_0^\ell$ and $\mathcal{S}_1^\ell$ for $\ell = 0, 1, 2, 3$ with the bits ordered from right to left.

There's just one missing thing in order to have a full information-theory representation of modern transmission systems, which is **interleaving**, which generally concerns shuffling either symbols or bits in an invertible manner. Although symbol-level interleaving would suffice to break off bursts of poor channel conditions, bit-level interleaving has the added advantage of shuffling also the bits contained in a given symbol; if the interleaving were deep enough to push any bit dependencies beyond the confines of each codeword, then the gap between $\sum_k \mathcal{I}(b_k; y|\boldsymbol{s})$ would be closed. It's true that interleaving has no effect when $N \to \infty$ and will thus have no effect on the mutual information calculations, but it's also true that it will improve the performance of actual codes with finite N.

The usage of **binary coding/decoding**, **bit-level interleaving**, **constellation mappers/soft demappers** gives the so-called **Bit-Interleaved Coded Modulation (BICM)** architecture. BICM is information-theoretically equivalent to signal-space coding for BPSK and Gray-Mapped QPSK. For higher-order constellations, even if the dependencies among same symbol bits are not fully eradicated by interleaving and the receiver ignores them, it is only slightly inferior.

In class:

What we usually do is that we have the input bits and we do **channel coding**; normally, channel coding is a **binary coding**.

Interleaver: sometimes the channel does errors in bursts, (it might be in a bad condition, ...); in that situation, the decoder cannot work as we'd have a long string of errors. What we do is to scramble the bits; if we have a matrix of memory, we write the values in rows

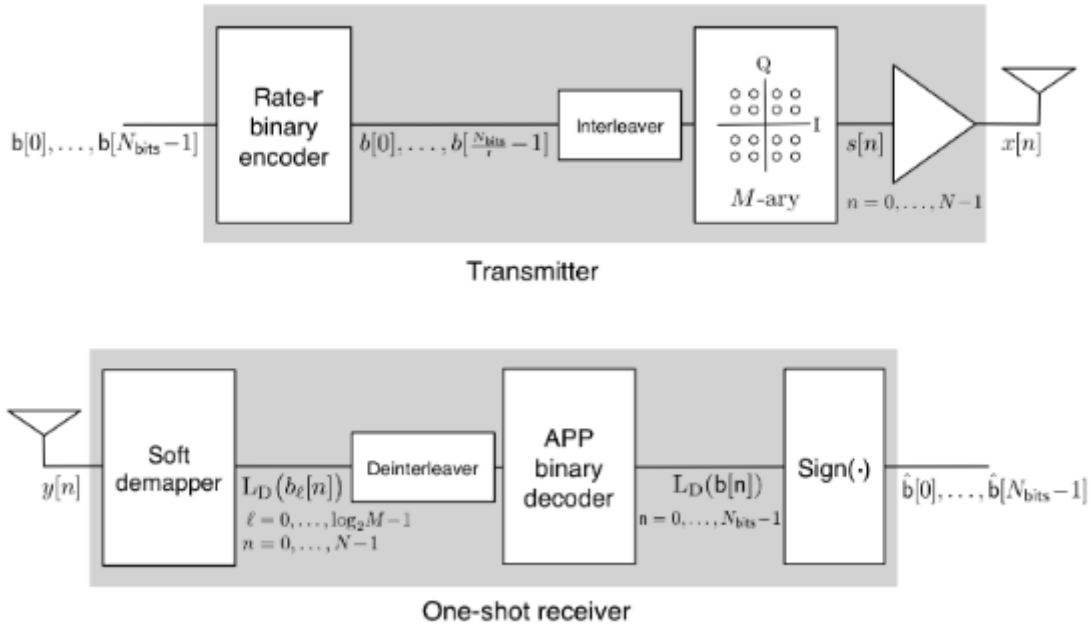


Figure 2.3: Top: BICM Transmitter Chain. Bottom: BICM Receiver Chain with Soft Demapper and Decoder.

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}$$

We read:

$$\begin{aligned} &\longrightarrow && 1, 2, 3, 4, \\ &\longleftarrow && 8, 7, 6, 5, \\ &\longrightarrow && 9, 10, 11, 12, \\ &\longleftarrow && 16, 15, 14, 13 \end{aligned}$$

And then:

$$\begin{aligned} &\downarrow && 1, 5, 9, 13, \\ &\downarrow && 2, 6, 10, 14, \\ &\downarrow && 3, 7, 11, 15, \\ &\downarrow && 4, 8, 12, 16 \end{aligned}$$

This way, bits which were close are now far apart, avoiding many consecutive errors; this is a simple interleaver! Then of course we'll do the reverse in the output.

There are many ways to implement interleavers, of course.

A positive **L-value** indicates that the bit in question is more likely to be a 1 than a 0, and vice versa for a negative value, with the magnitude indicating the confidence of the decision.
For a certain bit b, we define the **L-value** as

$$L(b) = \log \frac{P(b=1)}{P(b=0)}$$

such that if $L(b) > 0$, it's more likely that we have $b = 1$ and if $L(b) < 0$ it's more likely that we have $b = 0$. The bigger the magnitude, the more certain we are about our result.

At page 25 of the book, this concept is applied to an actual message, in the form of a **soft decoder**.

This thing we've just described (together with many other much more complex things handled from page 25 to page 30) is what is called an "A Posteriori Probability" decoder or "**APP**" decoder for short.

page 28

> Whichever the type of code, the sign of the L-values generated by an APP decoder for the message bits directly gives the MAP decisions

APP decoders are indeed really cool.

Let's say we transmit bits; we have a system which gives us values between -1 and 1; the closer we are to 1, the more we're sure that the value will be 1; the closer we are to -1, the more we're sure the value will be 0. We then use the sign function and we have back the binary values.

> From what I gathered, an **interleaver** is such to scrumble bits in order to remove dependency between bits.

The rest: didn't get it at all :D

Chapter 3

Week 3-9 November

3.1 Spectral efficiency for finite-length codewords

We can reach the Shannon bound only by pushing N to infinity; only if we're sending a *really* long codeword can we reach the capacity. We'd need to wait a long time (an *infinite* amount of time, I think) to fill that codeword and then we'd have to wait lots of latency in order to receive such a large signal.

We can show that by using small (non infinite!) N values, we don't reach the capacity and we can indeed show to which spectral efficiency we actually get!

$$\frac{R}{B} = C - \sqrt{\frac{V}{N}} Q^{-1}(p_e) + \mathcal{O}\left(\frac{\log N}{N}\right). \tag{1.143}$$

How below we stay depends on N, V, p_e; we don't need to remember the capacity, but we do need to remember the fact that **it's lower when compared to the capacity**.

Takeaway points are:

1. If the **block length** N **increases**, we get closer to the **ideal** capacity
2. If the **probability of error decreases**, we get closer to the **ideal** capacity (since $p_e \downarrow \implies Q \uparrow \implies (1/Q) \downarrow$)
3. If the **information density has a lower variance** (totally intuitively, i.e. if input/output are more related), we get closer to the **ideal** capacity

V is the variance of the information density, defined as

$$V = \text{var}[i(s; y)]. \tag{1.144}$$

This is a really cool result, as it allows us to understand how far away we are from ideal capacity.

About the exam
The sections published on the website are the program for the exam.

3.2 MMSE estimation

Ref: Chapter_1_EE_381S_Spg_2019, 08.1_pp_3_56_A_primer_on_information_theory_and_MMSE_estimation, page 37.

Probably next week we'll receive an email about the 1st of November and about where the lesson has been moved.
18th, 19th of 20th of November (and even more), professor will be in Paris. We'll have 3 extra dates, probably wed 16:30-18:30. We'll be able to find all the information about this on elearning.

3.2.1 Intro to MMSE estimation, scalar estimator (and the properties an estimator should have)

Today we'll do Minimum Mean Squared Error Estimation: MMSE estimation. This is something between telecommunications and control engineering. It goes back to Gauss and friends, but we've had much progress in the 20th century after the world war; the two major achievements are the Wieman and Calman filter. Without the Calman filter, we wouldn't have gone to the moon.

We'll go over different MMSE estimators; usually **estimating means we want to estimate a parameter**; here we have a transmitter and at the receiver we want to estimate the transmitted signal. This is the idea of this. This is not the only estimator though (we've seen MAP, ML, MD, ...). The last part of the chapter shows that the mutual information can be derived, instead of the usual way we've seen till now, as a derivate of what we'll call information MSE.

Our problem is to estimate a signal in noise

$$y = \sqrt{\rho}s + z \ ;$$

we have the observation y, we have $\sqrt{\rho}$, where ρ is the SNR and then we'll have z, Gaussian noise and s, unknown. We have some statistical information, such as:

- Posterior probability, probability density of s given y, $f_{s|y}(\cdot)$
- We have the likelihood function: we know the probability density of y given s, $f_{y|s}(\cdot)$
- Sometimes we have even the probability density of s, $f_s(\cdot)$!

Estimation problem, solution $\hat{s}(y)$
We want to find the solution $\hat{s}(y)$ which minimizes the MSE, $\mathbb{E}[\|s - \hat{s}(y)\|^2]$.

We can see this as a statement, but we'll do exercise 1.34 about this; the solution is

$$\hat{s}(y) = \mathbb{E}[s|y] \ . \tag{1.151}$$

When we make an estimator, we first of all need to check that the estimator is **unbiased**, aka that on average it gives me the actual quantity that I want to estimate: "If I estimate

many times, I should get my estimate!".

The thermometer is kind of estimating the temperature; the thermometer might be biased, though. The termometer might give always 1 degree more than the temperature. A good estimator should on average give a temperature of 24 if the temperature actually is 24.

This is **unbiased** in the sense that

$$\mathbb{E}[\hat{s}(y)] = \mathbb{E}[\mathbb{E}[s|y]] \overset{*}{=} \mathbb{E}[s] \ . \tag{1.153}$$

$*$ comes from the theorem of total expectation, which, by definition is such that

$$\mathop{\mathbb{E}}_{y}[\mathop{\mathbb{E}}_{s}[s|y]] = \mathop{\mathbb{E}}_{s}[s] \ .$$

The estimator **must satisfy orthogonality**; if we multiply the error by **any function** of the observation, it must be equal to 0:

$$\mathbb{E}[g(y^*)(s - \hat{s}(y))] = 0 \quad \forall \text{ function } g(\cdot) \ .$$

We're conjugating y; at the same time, g can contain a conjugate, so it's whatever: the conjugate is irrelevant. The conjugate will be useful when we'll manipulate the problem later.

Let's give an euristic interpretation of orthogonality: if we have a plane with two vectors (y, z, at least I think) and a vector going out of the plane (s); we want to give the best estimation of the vector which is going out of the plane:

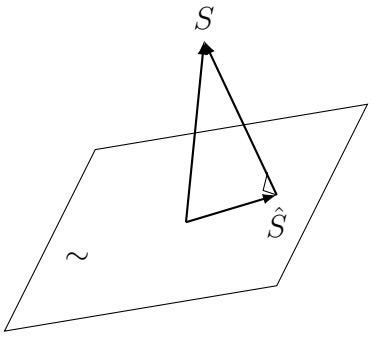


Figure 3.1: Interpretazione geometrica dell'ortogonalità: l'errore di stima è ortogonale al piano delle osservazioni.

We'll estimate the vector which is out of the plane, we'll have that our estimate is the projection of the vector on the plane and the error will be orthogonal to the plane (as can be seen in the figure).

We'll also have that the error is also orthogonal to $\hat{s}$ and to what it's available.

And what actually is the MMSE (until now we've only seen the estimator!)? It's the actual error we're making:

$$\text{MMSE}(\rho) = \mathbb{E}[\|s - \mathbb{E}[s|y]\|^2]. \tag{1.156}$$

In this equation it's important to look at the fact that the MMSE is only dependent on ρ.

3.2.2 MMSE estimator for a Complex Gaussian scalar through a channel

Let's say we have $y = \sqrt{\rho}s + z$ with

$$z \sim \mathcal{N}_{\mathbb{C}}(0,1) \implies y|s \sim \mathcal{N}_{\mathbb{C}}(\sqrt{\rho}s, 1) \implies f_{y|s} = \frac{1}{\pi}e^{-|y-\sqrt{\rho}s|^2}$$

We fix s; we can try and see what $y|s$ is: it's a complex Gaussian with mean $\sqrt{\rho}s$ and variance 1.

Let's look at the solution:

$$\begin{aligned}
\hat{s}(y) &= \mathbb{E}[s|y=y] \\
&= \int s f_{s|y}(s|y)\,\mathrm{d}s \\
&= \int \frac{s f_{y|s}(y|s) f_s(s)}{\int f_{y|s}(y|s) f_s(s)\,\mathrm{d}s}\,\mathrm{d}s \\
&= \frac{\int s f_{y|s}(y|s) f_s(s)\,\mathrm{d}s}{\int f_{y|s}(y|s) f_s(s)\,\mathrm{d}s} \\
&= \frac{\int s e^{-|y-\sqrt{\rho}s|^2} f_s(s)\,\mathrm{d}s}{\int e^{-|y-\sqrt{\rho}s|^2} f_s(s)\,\mathrm{d}s}
\end{aligned}$$

We go from $f_{s|y}$ to $f_{y|s}$ by using the Bayes theorem.
We can also have f_y as

$$f_y(Y) = \int f_{y,s}(Y,S)\,\mathrm{d}S = \int f_{y|s}(Y|S) f_s(S)\,\mathrm{d}S\ ,$$

from the marginalization rules.

Then specifically, for Gaussian signals,

$$\hat{s}(y) = \frac{\int s e^{-|y-\sqrt{\rho}s|^2} e^{-|s|^2}\,\mathrm{d}s}{\int e^{-|y-\sqrt{\rho}s|^2} e^{-|s|^2}\,\mathrm{d}s} \tag{1.167}$$

$$= \frac{\int s e^{-|\frac{|y|^2}{1+\rho} - \sqrt{1+\rho}s - \frac{\sqrt{\rho}}{1+\rho}y|^2}\,\mathrm{d}s}{\int e^{-|\frac{|y|^2}{1+\rho} - \sqrt{1+\rho}s - \frac{\sqrt{\rho}}{1+\rho}y|^2}\,\mathrm{d}s} \tag{1.168}$$

$$= \frac{\int s e^{-|s - \frac{\sqrt{\rho}}{1+\rho}y|^2/\frac{1}{1+\rho}}\,\mathrm{d}s}{\int e^{-|s - \frac{\sqrt{\rho}}{1+\rho}y|^2/\frac{1}{1+\rho}}\,\mathrm{d}s} \tag{1.169}$$

which can be notices to be the mean over the PDF of a complex gaussian distribution of

$$s' \sim \mathcal{N}_{\mathbb{C}}\left(\frac{\sqrt{\rho}}{1+\rho}y, \frac{1}{1+\rho}\right)\ ,$$

and we can then rewrite the values in (1.169) as

$$= \frac{\mathbb{E}[s']}{1} = \frac{\sqrt{\rho}}{1+\rho}y\ .$$

About the passages from (1.167)

To simplify the exponent in both numerator and denominator:

$$-|y-\sqrt{\rho}s|^2-|s|^2$$

We expand:

$$|y-\sqrt{\rho}s|^2=|y|^2-2\Re(y^*\sqrt{\rho}s)+\rho|s|^2$$

So,

$$-|y-\sqrt{\rho}s|^2-|s|^2=-|y|^2+2\Re(y^*\sqrt{\rho}s)-(1+\rho)|s|^2\,.$$

Our aim now is to try and isolate s from y; we then consider that for $a,b\in\mathbb{R}$,

$$|as+by|^2=a^2|s|^2+2ab\Re(y^*s)+b^2|y|^2\,;$$

if we look at the previous expression (forgetting for now of the sign flip), we can impose

$$a=\sqrt{1+\rho}\,,$$

from which follows that

$$ab=-\sqrt{\rho}\implies b=-\frac{\sqrt{\rho}}{\sqrt{1+\rho}}\,,$$

giving that

$$-|y-\sqrt{\rho}s|^2-|s|^2=-a^2|s|^2+2ab\Re(y^*s)-|y|^2:$$

we're missing a $-b^2|y|^2$ term! Then we sum and subtract $-b^2|y|^2$, getting

$$= -a^2|s|^2 + 2ab\Re(y^*s) - b^2|y|^2 + b^2|y|^2 - |y|^2 \,,$$

which we can then collect as

$$= -|as + by|^2 + (b^2 - 1)|y|^2 \,.$$

Since the term $(b^2-1)|y|^2$ will be in both exponentials, we can just cancel it out and forget about it. We're then left with

$$-|as+by|^2 = -\left|\sqrt{1+\rho}s - \frac{\sqrt{\rho}}{\sqrt{1+\rho}}y\right|^2 = -\left|s - \frac{\sqrt{\rho}}{1+\rho}y\right|^2 \Big/ \frac{1}{1+\rho} :$$

our initial expression will now be

$$\frac{\int_{-\infty}^{+\infty} s \exp\left(-\left|s - \frac{\sqrt{\rho}}{1+\rho}y\right|^2 \Big/ \frac{1}{1+\rho}\right) ds}{\int_{-\infty}^{+\infty} \exp\left(-\left|s - \frac{\sqrt{\rho}}{1+\rho}y\right|^2 \Big/ \frac{1}{1+\rho}\right) ds} \,,$$

where the top is the expectation of $s \sim \mathcal{N}_{\mathbb{C}}\left(\frac{\sqrt{\rho}}{1+\rho}y, \frac{1}{1+\rho}\right)$ and the bottom is the integral of the whole PDF of such RV; we'll then have that this is

$$= \frac{\frac{\sqrt{\rho}}{1+\rho}y}{1} = \frac{\sqrt{\rho}}{1+\rho}y \,.$$

If we suppose $s \sim \mathcal{N}_{\mathbb{C}}(0,1)$, using the previous example and some calculus we find that the MMSE estimator for that s is

$$\hat{s}(y) = \frac{\sqrt{\rho}}{1+\rho}y \,. \tag{1.172}$$

This is good because we'll often suppose that the input is Gaussian. This is a linear transformation of the observation!

Importance of Gaussian results
Whichever type of result which assumes $s \sim \mathcal{N}_{\mathbb{C}}(0,1)$ is extremely important in that most of the information theory we'll derive for capacity and similar will assume a Gaussian input.

If I have to calculate the best estimation and we have a Gaussian input, we can take the input, multiply it by the quantity which we've found and we'll have our estimator.

The minimum of the mean squared error is actually

$$\text{MMSE}(\rho) = \mathbb{E}\left[\left|s - \frac{\sqrt{\rho}}{1+\rho}y\right|^2\right] \tag{1.173}$$

$$= \mathbb{E}[|s|^2] - 2\frac{\sqrt{\rho}}{1+\rho}\Re[\mathbb{E}[ys^*]] + \frac{\rho}{(1+\rho)^2}\mathbb{E}[|y|^2] \tag{1.174}$$

$$= \mathbb{E}[|s - \hat{s}(y)|^2] = \frac{1}{1+\rho}\,, \tag{1.175}$$

where (1.175) follows from the fact that $\mathbb{E}[\|s\|^2] = 1, \mathbb{E}[ys^*] = \sqrt{\rho}, \mathbb{E}[\|y\|^2] = 1 + \rho$.

This means that if ρ is big, the error becomes smaller, which is what we expect. If we have low noise, we expect to have no error. With a SNR of 0 (no signal, all error), we have an error of 1.

3.2.3 Vectorial estimator

Let's reformulate for vectors:

$$\boldsymbol{y} = \sqrt{\rho}\boldsymbol{A}\boldsymbol{s} + \boldsymbol{z}, \tag{1.186}$$

where we know that $\boldsymbol{A}$ is a matrix (not necessarily a square matrix though!). We have another assumption, that the SNR is the same for every signal in our vector.

MMSE estimator, MSE matrix for a random vector

The MMSE estimator is

$$\hat{\boldsymbol{s}}(\boldsymbol{y}) = \mathbb{E}[\boldsymbol{s}|\boldsymbol{y}]\,. \tag{1.187}$$

The MSE matrix is

$$\boldsymbol{E} = \mathbb{E}[(\boldsymbol{s} - \hat{\boldsymbol{s}}(\boldsymbol{y}))(\boldsymbol{s} - \hat{\boldsymbol{s}}(\boldsymbol{y}))^*]\,. \tag{1.188}$$

The matrix $\boldsymbol{E}$ is a matrix which in the main diagonal will have for a vector $\boldsymbol{s} = (s_1, s_2, \ldots, s_n)$ that the j, j element will give $\boldsymbol{E}_{jj} = \text{MSE}(s_j)$.

3.2.4 MMSE estimator for a Complex Gaussian vector representing a channel

Example of estimation of a vector Gaussian: we have $\boldsymbol{s} \sim \mathcal{N}_{\mathbb{C}}(\boldsymbol{0}, \boldsymbol{R_s})$ and $\boldsymbol{z} \sim \mathcal{N}_{\mathbb{C}}(\boldsymbol{0}, \boldsymbol{I})$, for a channel expressed as $\boldsymbol{y} = \sqrt{\rho}\boldsymbol{A}\boldsymbol{s} + \boldsymbol{z}$.

We can find the estimator (by using (1.187)) as

$$\hat{\boldsymbol{s}}(\boldsymbol{y}) = \sqrt{\rho}\boldsymbol{R_s}\boldsymbol{A}^*(\boldsymbol{I} + \rho\boldsymbol{A}\boldsymbol{R_s}\boldsymbol{A}^*)^{-1}\boldsymbol{y}\,. \tag{1.189}$$

> Just like in the scalar case, the estimator is still linear.

We also have to calculate the error matrix; it will be

$$\begin{aligned}\boldsymbol{E} &= \mathbb{E}[\boldsymbol{s}\boldsymbol{s}^*] - \mathbb{E}[\hat{\boldsymbol{s}}\hat{\boldsymbol{s}}^*] - \mathbb{E}[\hat{\boldsymbol{s}}\boldsymbol{s}^*] + \mathbb{E}[\hat{\boldsymbol{s}}\hat{\boldsymbol{s}}^*] \\ &= \boldsymbol{R_s} - 2\rho\boldsymbol{R_s}\boldsymbol{A}^*(\boldsymbol{I} + \rho\boldsymbol{A}\boldsymbol{R_s}\boldsymbol{A}^*)^{-1}\boldsymbol{A}\boldsymbol{R_s} \\ &\quad + \rho\boldsymbol{R_s}\boldsymbol{A}^*(\boldsymbol{I} + \rho\boldsymbol{A}\boldsymbol{R_s}\boldsymbol{A}^*)^{-1}(\boldsymbol{I} + \rho\boldsymbol{A}\boldsymbol{R_s}\boldsymbol{A}^*)(\boldsymbol{I} + \rho\boldsymbol{A}\boldsymbol{R_s}\boldsymbol{A}^*)^{-1}\boldsymbol{A}\boldsymbol{R_s} \\ &= \boldsymbol{R_s} - \rho\boldsymbol{R_s}\boldsymbol{A}^*(\boldsymbol{I} + \rho\boldsymbol{A}\boldsymbol{R_s}\boldsymbol{A}^*)^{-1}\boldsymbol{A}\boldsymbol{R_s}\,.\end{aligned}$$

We can rearrange those matrices in an elegant manner:

$$\boldsymbol{E} = (\boldsymbol{R}_{\boldsymbol{s}}^{-1} + \rho \boldsymbol{A}^* \boldsymbol{A})^{-1} .$$

This follows from the matrix inversion lemma (with $\boldsymbol{A} \to \boldsymbol{R}_{\boldsymbol{s}}^{-1}, \boldsymbol{B} \to \rho \boldsymbol{A}^*, \boldsymbol{C} \to \boldsymbol{I}, \boldsymbol{D} \to \boldsymbol{A}$),

Matrix inversion lemma
We have that

$$(\boldsymbol{A} + \boldsymbol{BCD})^{-1} = \boldsymbol{A}^{-1} - \boldsymbol{A}^{-1}\boldsymbol{B}(\boldsymbol{C}^{-1} + \boldsymbol{DA}^{-1}\boldsymbol{B})^{-1}\boldsymbol{DA}^{-1} .$$

This means that we only want $\boldsymbol{A}, \boldsymbol{C}$ to be invertible!

Thanks to the matrix inversion lemma, we find something which is pretty similar to what we had before,

$$\begin{aligned} \mathrm{MMSE}(\rho) &= \mathbb{E}\left[\left| s - \frac{\sqrt{\rho}}{1+\rho} y \right|^2\right] && (1.173) \\ &= \mathbb{E}[|s|^2] - 2\frac{\sqrt{\rho}}{1+\rho}\Re[\mathbb{E}[y s^*]] + \frac{\rho}{(1+\rho)^2}\mathbb{E}[|y|^2] && (1.174) \\ &= \mathbb{E}[|s - \hat{s}(y)|^2] = \frac{1}{1+\rho} , && (1.175) \end{aligned}$$

.

Skipped
Wrt the book, we're only missing the expansion of the error matrix for low-SNR.

3.3 I-MMSE relationship in Gaussian noise

A relation has been found around 20 years ago between information theory (I) and estimation theory (MMSE)!

Information-MMSE scalar relationship
Referring to the relation $y = \sqrt{\rho} s + z$ with s, z independent and $z \sim \mathcal{N}_{\mathbb{C}}(0,1)$, by using $\mathcal{I}(\rho) = I(s;y)$, we have the equivalence

$$\frac{1}{\log_2 e}\frac{d}{d\rho}\mathcal{I}(\rho) = \mathrm{MMSE}(\rho) \quad (1.196)$$

or

$$\frac{1}{\log_2 e}\mathcal{I}(\rho) = \int_0^\rho \mathrm{MMSE}(\xi)\, d\xi . \quad (1.197)$$

If we check this relation on the Complex Gaussian scalar case, we get that

$$\begin{aligned} \mathcal{I}(\rho) &= \log_2(1+\rho) && (1.198) \\ \mathrm{MMSE}(\rho) &= \frac{1}{1+\rho} , && (1.199) \end{aligned}$$

which satisfies the equation.

Information-MMSE vector relationship
What about the vector case? We use the gradient operator!

$$\frac{1}{\log_2 e}\nabla_{\boldsymbol{A}} I(\boldsymbol{s}; \sqrt{\rho}\boldsymbol{A}\boldsymbol{s} + \boldsymbol{z}) = \rho \boldsymbol{A}\boldsymbol{E} \, .$$

Let's test this relation for complex Gaussian vectors:

Example 1.35 (I-MMSE relationship for a complex Gaussian vector) As established in Example 1.13, when the noise is $\boldsymbol{z} \sim \mathcal{N}_{\mathbb{C}}(\boldsymbol{0}, \boldsymbol{I})$ while $\boldsymbol{s} \sim \mathcal{N}_{\mathbb{C}}(\boldsymbol{0}, \boldsymbol{R_s})$,

$$\mathcal{I}(\boldsymbol{s}; \sqrt{\rho}\boldsymbol{A}\boldsymbol{s} + \boldsymbol{z}) = \log_2 \det(\boldsymbol{I} + \rho \boldsymbol{A}\boldsymbol{R_s}\boldsymbol{A}^*) \tag{1.208}$$
$$= \log_2 \det(\boldsymbol{I} + \rho \boldsymbol{A}^*\boldsymbol{A}\boldsymbol{R_s}) \tag{1.209}$$

and, applying the expression for the gradient of a log-determinant function given in Appendix D, we obtain

$$\frac{1}{\log_2 e}\nabla_{\boldsymbol{A}} I(\boldsymbol{s}; \sqrt{\rho}\boldsymbol{A}\boldsymbol{s} + \boldsymbol{z}) = \rho \boldsymbol{A}\boldsymbol{R_s}(\boldsymbol{I} + \rho \boldsymbol{A}^*\boldsymbol{A}\boldsymbol{R_s})^{-1} \tag{1.210}$$
$$= \rho \boldsymbol{A}(\boldsymbol{R_s}^{-1} + \rho \boldsymbol{A}^*\boldsymbol{A})^{-1} \, , \tag{1.211}$$

which indeed equals $\rho\boldsymbol{A}\boldsymbol{E}$ with $\boldsymbol{E} = (\boldsymbol{R_s}^{-1} + \rho\boldsymbol{A}^*\boldsymbol{A})^{-1}$ as determined in Example 1.31 for a complex Gaussian vector.

We can conclude that the expression also works for vectors!

3.4 LMMSE for random variables

Disclaimer
This paragraph is not to be confused with the future paragraph **LMMSE equalization**, which instead discusses how to use the contents of this paragraph in order to equalize the output of a noisy channel, so to recover the original signal.

We can find a different type of estimator which can be used in order to **estimate when quantities which we don't know are involved** (i.e.: transformations made by the channel): it is a linear estimator which will prove to be more robust and versatile, but also inferior, when compared with a conditional-mean estimator (a conditional mean estimator is an estimator between the ones which have been discussed until now), which we'll refer to as the "Linear MMSE estimator" or "LMMSE estimator".

We can use wiener filtering if we do some operations beforehand, but only to stationary filters.
If the process is not stationary, we need to use Calemann filters.

The LMMSE problem sees us finding a vector $\boldsymbol{b}^{\text{MMSE}}$ and matrix $\boldsymbol{W}^{\text{MMSE}}$ such that we minimize the mean squared error between $\boldsymbol{s}$ and our estimate

$$\hat{\boldsymbol{s}} = \boldsymbol{W}^{\text{MMSE}}\boldsymbol{y} + \boldsymbol{b}^{\text{MMSE}} \, . \tag{1.223}$$

The term $\boldsymbol{b}$ is used **in order to ensure that the estimator is unbiased**; it is generally set to $\mathbf{0}$, since we generally assume we're using zero-mean signals.

The estimator is simply

$$\boldsymbol{W}^{\text{MMSE}} = \boldsymbol{R}_{\boldsymbol{y}}^{-1}\boldsymbol{R}_{\boldsymbol{ys}} \ . \tag{1.224}$$

In order to find (1.224), we consider that we want to minimize the mean-squared error; **we then write the mean-square error on the estimation of the jth entry of $\boldsymbol{s}$ via a generic linear filter $\boldsymbol{W}$ (s.t. $\hat{\boldsymbol{s}} = \boldsymbol{W}^*\boldsymbol{y}$) as**

$$\begin{aligned}
\mathbb{E}[|[\boldsymbol{s} - \hat{\boldsymbol{s}}]_j|^2] &= \mathbb{E}[|[\boldsymbol{s} - \boldsymbol{W}^*\boldsymbol{y}]_j|^2] && (1.225)\\
&= \mathbb{E}[|s_j - \boldsymbol{w}_j^*\boldsymbol{y}|^2] && (1.226)\\
&= \mathbb{E}[|s_j|^2] - \mathbb{E}[\boldsymbol{w}_j^*\boldsymbol{y}s_j^*] - \mathbb{E}[s_j\boldsymbol{y}^*\boldsymbol{w}_j] + \mathbb{E}[\boldsymbol{w}_j^*\boldsymbol{y}\boldsymbol{y}^*\boldsymbol{w}_j] && (1.227)
\end{aligned}$$

where $\boldsymbol{w}_j \triangleq [\boldsymbol{W}]_{:,j}$ is the jth column of $\boldsymbol{W}$, which is the only part interested in estimating the entry $s_j \triangleq [\boldsymbol{s}]_j$.

Then if we consider the gradient wrt $\boldsymbol{w}_j$, we see that, since

$$\begin{aligned}
\nabla_{\boldsymbol{x}}(\boldsymbol{x}^*\boldsymbol{r}) &= \boldsymbol{r} && \text{(D.5)}\\
\nabla_{\boldsymbol{x}}(\boldsymbol{r}^*\boldsymbol{x}) &= \mathbf{0} && \text{(D.6)}\\
\nabla_{\boldsymbol{x}}(\boldsymbol{x}^*\boldsymbol{R}\boldsymbol{x}) &= \boldsymbol{R}\boldsymbol{x} \ , && \text{(D.7)}
\end{aligned}$$

$$\begin{aligned}
\nabla_{\boldsymbol{w}_j}\mathbb{E}\left[|[\boldsymbol{s} - \hat{\boldsymbol{s}}]_j|^2\right] &= -\mathbb{E}\left[\boldsymbol{y}s_j^*\right] + \mathbb{E}\left[\boldsymbol{y}\boldsymbol{y}^*\boldsymbol{w}_j\right]\\
&= -\mathbb{E}\left[\boldsymbol{y}(s_j - \boldsymbol{w}_j^*\boldsymbol{y})^*\right]\\
&= -\mathbb{E}\left[\boldsymbol{y}[\boldsymbol{s} - \hat{\boldsymbol{s}}]_j^*\right] \ ,
\end{aligned}$$

which, once equated to zero (in order to minimize it) gives

$$\begin{aligned}
-\mathbb{E}\left[\boldsymbol{y}(\boldsymbol{s} - \boldsymbol{W}^{\text{MMSE}^*}\boldsymbol{y})^*\right] &= \mathbb{E}[\boldsymbol{y}\boldsymbol{y}^*]\boldsymbol{W}^{\text{MMSE}} - \mathbb{E}[\boldsymbol{y}\boldsymbol{s}^*]\\
&= \mathbf{0}
\end{aligned}$$

and thus that

$$\boldsymbol{R}_{\boldsymbol{y}}\boldsymbol{W}^{\text{MMSE}} - \boldsymbol{R}_{\boldsymbol{ys}} = 0 \ , \tag{1.233}$$

which then easily leads to (1.224).

About the concept of unbiased

For the concept of unbiased, refer to

> This is **unbiased** in the sense that
>
> $$\mathbb{E}[\hat{s}(y)] = \mathbb{E}[\mathbb{E}[s|y]] \overset{*}{=} \mathbb{E}[s] \ . \tag{1.153}$$

About the inversion

Question: is the -1 over the matrix a problem? An autocorrelation matrix is semidefinite positive. If the vectors are proper (and they are thanks to our assumption), it is semidefinite positive. This means that if we take the matrix and multiply it by two vectors, we get something positive:

$$\boldsymbol{v}^* \begin{bmatrix} & \boldsymbol{R} & \end{bmatrix} \boldsymbol{v} > 0$$

From algebra we know that semidefinite positive matrices are always invertible. Multiplying a semidefinite positive matrix by a positive value keeps the property of semidedefinite positivity.

We always need to pay attention when we see a -1 over a matrix!

Let's then derive the **MMSE matrix for the LMMSE**:

$$\begin{aligned}
\boldsymbol{E} &= \mathbb{E}[(\boldsymbol{s} - \hat{\boldsymbol{s}}(\boldsymbol{y}))(\boldsymbol{s} - \hat{\boldsymbol{s}}(\boldsymbol{y}))^*] && (1.241)\\
&= \boldsymbol{R}_{\boldsymbol{s}} - \boldsymbol{R}_{\boldsymbol{s}\hat{\boldsymbol{s}}} - \boldsymbol{R}^*_{\hat{\boldsymbol{s}}\boldsymbol{s}} + \boldsymbol{R}_{\hat{\boldsymbol{s}}} && (1.242)\\
&= \boldsymbol{R}_{\boldsymbol{s}} - \boldsymbol{R}_{\hat{\boldsymbol{s}}} - \boldsymbol{R}^*_{\hat{\boldsymbol{s}}} + \boldsymbol{R}_{\hat{\boldsymbol{s}}} && \\
&= \boldsymbol{R}_{\boldsymbol{s}} - \boldsymbol{R}^*_{\boldsymbol{y}\boldsymbol{s}}\boldsymbol{R}_{\boldsymbol{y}}^{-1}\boldsymbol{R}_{\boldsymbol{y}\boldsymbol{s}} \ , && (1.243)
\end{aligned}$$

with (1.243) following from the facts that:

- $$\begin{aligned}
\boldsymbol{R}_{\hat{\boldsymbol{s}}} &= \mathbb{E}[\hat{\boldsymbol{s}}\hat{\boldsymbol{s}}^*] && (1.234)\\
&= \boldsymbol{W}^{\mathrm{MMSE}*}\mathbb{E}[\boldsymbol{y}\boldsymbol{y}^*]\boldsymbol{W}^{\mathrm{MMSE}} && (1.235)\\
&= \boldsymbol{R}^*_{\boldsymbol{y}\boldsymbol{s}}\boldsymbol{R}_{\boldsymbol{y}}^{-1}\boldsymbol{R}_{\boldsymbol{y}}\boldsymbol{R}_{\boldsymbol{y}}^{-1}\boldsymbol{R}_{\boldsymbol{y}\boldsymbol{s}} \qquad \text{def. of } \boldsymbol{W}^{\mathrm{MMSE}} && (1.236)\\
&= \boldsymbol{R}^*_{\boldsymbol{y}\boldsymbol{s}}\boldsymbol{R}_{\boldsymbol{y}}^{-1}\boldsymbol{R}_{\boldsymbol{y}\boldsymbol{s}} \ . && (1.237)
\end{aligned}$$

- $$\begin{aligned}
\boldsymbol{R}_{\hat{\boldsymbol{s}}\boldsymbol{s}} &= \mathbb{E}[\boldsymbol{W}^{\mathrm{MMSE}*}\boldsymbol{y}\boldsymbol{s}^*] && (1.238)\\
&= \boldsymbol{R}^*_{\boldsymbol{y}\boldsymbol{s}}\boldsymbol{R}_{\boldsymbol{y}}^{-1}\boldsymbol{R}_{\boldsymbol{y}\boldsymbol{s}} \qquad \text{def. of } \boldsymbol{W}^{\mathrm{MMSE}} && (1.239)\\
&= \boldsymbol{R}_{\hat{\boldsymbol{s}}} \qquad \text{from (1.237)} && (1.240)
\end{aligned}$$

3.5 Extension (from random variables to random processes)

If we extend this to random processes, the relation in that case is

$$y[n] = \sqrt{\rho}s[n] + z[n] \ .$$

We'll need to generalize from random variable to random process. In order to find LMMSE, we can use a causal or a noncausal filter:

$$\text{MMSE} = 1 - \int_{-1/2}^{1/2} \frac{\rho S(\nu)}{1+\rho S(\nu)}\, d\nu \qquad \text{noncausal}$$

$$\text{MMSE} = \left[\exp\left(\int_{-1/2}^{1/2} \log_e(1+\rho S(\nu))\, d\nu\right) - 1\right] \qquad \text{causal}$$

Causal means that we can't use the future to compute something now: we can't use future values of x, z, y to compute an estimate. In the real world we can only use causal filters.

About this being asked
I'm pretty positive this will never be asked.

$S(\cdot)$ is the power spectrum of $s[\cdot]$, btw.

Problem 1.34 (proof of the orthogonality principle and of the linear MMSE estimator)

Exercise time! Text:

1.34 Consider the transformation $y = \sqrt{\rho}s + z$.

(a) Prove that, for any arbitrary function $g(\cdot)$, $\mathbb{E}[g(y)(s - \mathbb{E}[s|y])] = 0$. This is the so-called *orthogonality principle*.

(b) Taking advantage of the orthogonality principle, prove that the MMSE estimate is given by $\hat{s}(y) = \mathbb{E}[s|y]$.

For what concerns question (a), we can exploit the total probability theorem, thanks to which here we can write

$$\begin{aligned}\mathbb{E}[g(y)(s - \mathbb{E}[s|y])] &= \mathbb{E}_y[\mathbb{E}[g(y)(s - \mathbb{E}[s|y])|y]] \\ &= \mathbb{E}_y[g(y)(\mathbb{E}[s|y] - \mathbb{E}[s|y])] \\ &= \mathbb{E}_y[g(y) \cdot 0] = 0\end{aligned}$$

where the very first passage was one of the various applications of the total probability theorem.

Instead, for question (b), we know that the MMSE is $\mathbb{E}[|s - \hat{s}(y)|^2]$; we can minimize it through:

The expectation $\mathbb{E}[s|y]$ is a function of y; to search for the MMSE solution, we need to take this into account. Point **a** of the exercise is telling us about orthogonality between the error and *any* function of y. The intuition about this is the one which has been shown using vectors and planes.

Let's then express the MMSE. We use a trick by adding and subtracting $\mathbb{E}[s|y]$ inside the norm:

$$\mathbb{E}\left[|s-\hat{s}(y)|^2\right] = \mathbb{E}\left[|\underbrace{s-\mathbb{E}[s|y]}_{a}+\underbrace{\mathbb{E}[s|y]-\hat{s}(y)}_{b}|^2\right]$$

We know that for $a, b \in \mathbb{C}$, we have an equation for the modulo:

$$|a+b|^2 = |a|^2 + |b|^2 + 2\Re\{ab^*\}$$

Applying this to our expectation:

$$\begin{aligned} &= \mathbb{E}\left[|s-\mathbb{E}[s|y]|^2\right] + \mathbb{E}\left[|\mathbb{E}[s|y]-\hat{s}(y)|^2\right] \\ &+ 2\Re\left\{\mathbb{E}\left[(s-\mathbb{E}[s|y])(\underbrace{\mathbb{E}[s|y]-\hat{s}(y)}_{g(y)})^*\right]\right\} \end{aligned}$$

Let's analyze the cross-term (the $2\Re\{\dots\}$ part). The term $(\mathbb{E}[s|y]-\hat{s}(y))^*$ is a function of y, let's call it $g(y)^*$. From **before** (part a), we know that

$$\mathbb{E}[g(y)^*(s-\mathbb{E}[s|y])] = 0$$

So the entire cross-term cancels out!

We are left with:

$$= \underbrace{\mathbb{E}\left[|s-\mathbb{E}[s|y]|^2\right]}_{\text{Cannot change this}} + \underbrace{\mathbb{E}\left[|\mathbb{E}[s|y]-\hat{s}(y)|^2\right]}_{\geq 0 \text{ iff } \hat{s}(y)=\mathbb{E}[s|y]}$$

We can play on $\hat{s}(y)$ in order to minimize the MMSE: this happens for $\hat{s}(y) = \mathbb{E}[s|y]$, as this gets us only to be zero for the part in green (the second term).

we've thus proven the orthogonality principle.

3.6 Signal processing

Book reference: 08.2_pp_57_138_A_signal_processing_perspective.

3.6.1 Introduction

We'll look at chapter two, which looks at the DSP perspective; until now we've only looked at information theory and MSE theory; we also saw that the two theories can be related: these are the foundations!

Ref: Chapter_2_EE_381S_Spg_2019.

We need to study on the book oc: only there we will find all the details.

Highlights:

- Complex baseband signal model allows representing passband signals regardless of the actual carrier frequency
- Because the transmit signal is bandlimited, a complete system model can be obtained in discrete-time, including the portion of the channel impacted by the transmit signal
- Several different standard MIMO signal models are used, sometimes vectorized depending on the problem at hand
- Precoding is a structured means to create a correlated transmit signal, and can be normalized in different ways depending on the signal constraints
- OFDM and SC-FDE have an equalization structure that is independent of the memory in the channel, at the expense of some overheads
- Channels are estimated using pilots, and may exploit channel statistics

We'll see that channels are estimated using what we'll call *pilots*.

3.7 Passband signal, in-phase/in-quadrature components

We normally transmit waveforms characterized by a signal of voltage $x_p(t)$, applied to the antenna. This signal is generally passband, in the sense that in the frequency domain most of its energy is concentrated around a **carrier frequency** f_c.

Let's suppose that $x_p(t)$ is an ideal passband signal (by ideal we mean that in the frequency domain it will be nonzero only over a bandwidth B centered on f_c): we'll have that $B \ll f_c$, which we'll refer to as the **narrowband condition**.

Under the narrowband condition, we can write

$$x_p(t) = A(t)\cos(2\pi f_c t + \phi(t)) \ ,$$

where:

- $A(t)$ is a magnitude function
- $\phi(t)$ is a phase function

Applying trigonometric identities to that equation, we can arrange it into

$$x_p(t) = \underbrace{A(t)\cos(\phi(t))}_{x_c(t)}\cos(2\pi f_c t) - \underbrace{A(t)\sin(\phi(t))}_{x_s(t)}\sin(2\pi f_c t) \ ,$$

where:

- $x_c(t)$ is called the *in-phase component*; it modulates a cosine carrier
- $x_s(t)$ is called the *in-quadrature component*; it modulates a sine carrier

Complex envelope of a passband signal

We'll now define the **complex envelope** or **complex baseband equivalent** of $x_p(t)$ as

$$x(t) = x_c(t) + jx_s(t) \ ;$$

this will give that the passband signal will be obtained as

$$x_p(t) = \sqrt{2}\Re\{x(t)e^{j2\pi f_c t}\} \ .$$

$x_p(t)$ is a physical quantity and it will thus be $\in \mathbb{R}$; $x(t)$, instead, will be $\in \mathbb{C}$.

In the frequency domain we'll have

$$x_p(f) = \frac{1}{\sqrt{2}}[x(f - f_c) + x^*(-f - f_c)] \ . \tag{2.5}$$

If $x_p(f)$ is bandlimited with passband bandwidth of B, then $x(f) = x_c(f) + jx_s(f)$ and its components x_c, x_s will all be bandlimited with baseband bandwidth $B/2$. Moreover, the two components are orthogonal thanks to the phase difference between the cosine and sine carriers.

The process of going from the baseband signal to the passband equivalent is called **upconversion**, while the opposite process is called **downconversion**.

Upconversion can be easily implemented as follows:

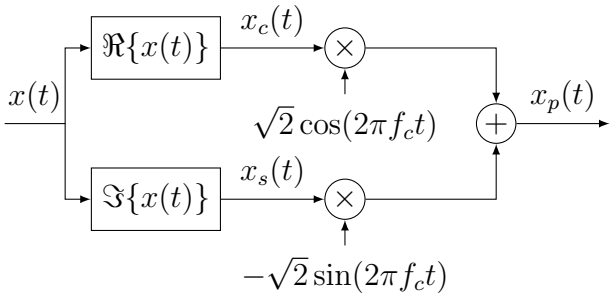


Figure 3.2: Conversion of a baseband signal to a passband signal.

Downconversion is a bit more involved. Let's say $y_p(t)$ is the signal at the receiver. First of all, the signal is bandpass-filtered in order to have it reject transmission on other frequency bands. Being bandpass, $y_p(t)$ will have a baseband equivalent $y(t) = y_c(t) + jy_s(t)$ such that, just like what we said about the sent signal, we'll have

$$y_p(t) = \sqrt{2}[y_c(t)\cos(2\pi f_c t) - y_s(t)\sin(2\pi f_c t)] \,. \tag{2.6}$$

There are ways to extrapolate $y_c(t)$ and $y_s(t)$ using pertinent low pass filters, which need to be realized while knowing the **carrier frequency** f_c: 08.2_pp_57_138_A_signal_processing_perspective, p.63.
f_c is generally not known precisely; estimating it is a matter tackled by "carrier frequency estimation".

The conversion to the baseband is mostly important since most of the processing is done on low frequency signals.

Wireless communications systems send and receive passband signals; those signals are not directly created in passband, though.

Bluetooth is at 5.4GHz; we don't have circuits at those frequencies, as they'd be super intense.

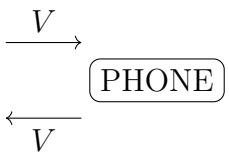
What we generally do is to take signals in baseband and then we scale them.

3.7.1 Random digression on why radio doesn't transmit on baseband and on ADSL

Why are radio signals not transmitted in baseband? ADSL transmits in baseband to avoid attenuation.

Why aren't radio signals transmitted as baseband? We need to go to **high frequencies if we don't want to make gigantic antennas**.

We studied radio; but what about ADSL? We'd have a twisted pair of cables coming to our house. Let's say we have a phone. On a cable we're sending voice, on another we're receiving voice; we do so by changing a certain voltage:

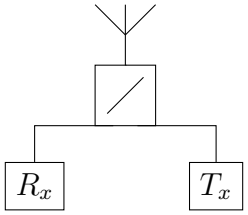


How is it that with just two cables we don't hear a distorted signal (i.e.: how is it that we don't hear our own voice)? We only have one voltage difference! We could implement FDMA, but it would be impractical! In order to synthesize a frequency, we'd need a crystal and a VCO (voltage controlled oscillator). What we actually do is use the *forchetta telefonica* (telephone hybrid); we have a circulator which is able to distinguish the incoming and outgoing signal: it cancels the outgoing voltage from the total, getting the incoming voltage as a result. The telephone hybrid allows us to go from 2 to 4 wires.

With ADSL we have something like FDMA; we have frequencies up to 4KHz and such (this generally applies to voices; instruments and such will have higher frequencies but we don't really care lol).

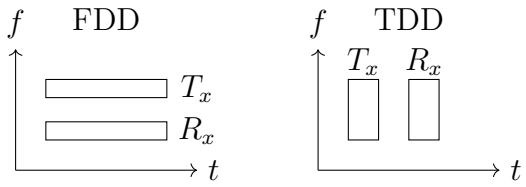
3.7.2 Random digression on FDD (Frequency Division Duplex) and TDD (Time Divison Duplex)

FDD and TDD. When we're transmitting with a radio in SISO, we have an antenna transmitting and an antenna receiving. When we're transmitting, we can imagine we have a switch which selects the transmitter; when we want to receive, we'll want to select the receiver:



We also need to be careful about not having contact between receiver and transmitter, as we'd destroy the receiver (which is designed in order to use much smaller voltages)!

FDD: Frequency Division Duplex; we transmit at the same time using different frequencies. **TDD: Time Division Duplex**; we have different time slots for transmitting and receiving.



The problem is that the received signal is much smaller than the transmitted; it's difficult to distinguish them.

Full duplex is now being researched: with full duplex, we use the same frequencies and we transmit in the same time slot.

3.8 Complex baseband channel response

The channel for wireless transmission can be modeled as a LTI channel.

If we have a description of the response of the channel to a passband signal, we denote it as

$$c_p(\tau) \ ,$$

such that we'll have

$$y_p(t) = (c_p * x_p)(t)$$

we want to find another **baseband representation of the channel response**, $c_b(\tau)$, such that

$$y(t) = [c_b * x](t) \ .$$

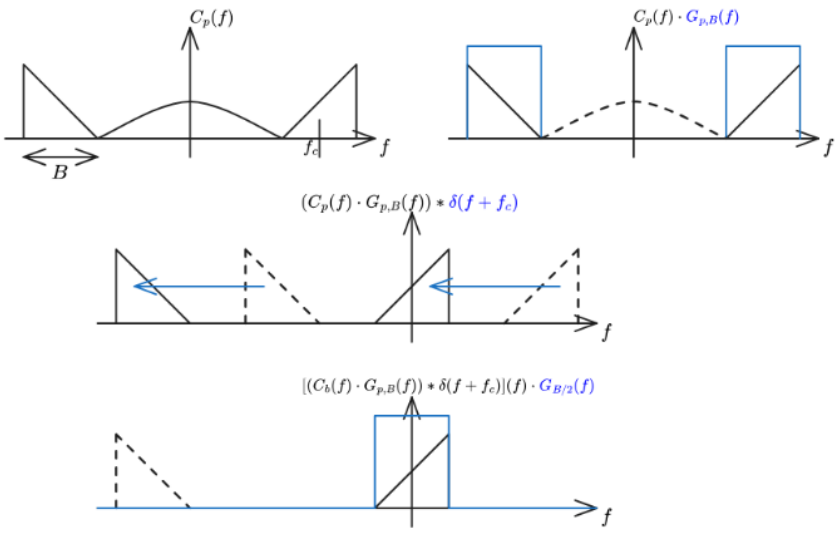


Figure 3.3: Operations to obtain the baseband channel response from the passband channel response.

About the time invariance: it's more generic to say that the channel is roughly constant over certain intervals of time and frequency.

We thus need to find an expression of c_b starting from c_p. Let's look at the various operations we want our baseband channel to implement:

Let's cover each step:

1. We have our channel response, which in the frequency domain might have extra elements on top of the part which interests us (a section of bandwidth B around the frequency f_c).
2. We select only the part of the response which interests us, by multiplying the values for negative and positive frequencies around f_c with two ideal rect functions

$$G_{p,B}(f) = \text{rect}\left(\frac{f - f_c}{B}\right) + \text{rect}\left(\frac{f + f_c}{B}\right) ,$$

which in the time domain will be

$$g_{p,B}(t) = \underbrace{B\text{sinc}(Bt)}_{\text{env}} \underbrace{2\cos(2\pi f_c t)}_{\text{shift}}$$

and will be convolved with $c_p(t)$:

$$(c_p(\cdot) * g_{p,B}(\cdot))(t) \ .$$

3. We shift the selected values towards the left, centering around 0 the part of the response which was around f_c; this is equal to convoluting with $\delta(f + f_c)$ or, in the time domain, multiplying by the corresponding oscillating signal:

$$e^{-j2\pi f_c t}(c_p(\cdot) * g_{p,B}(\cdot))(t) \ .$$

4. We only select the central component by multiplying our response with an ideal rect centered around 0:

$$G_{b,B/2}(f) = \text{rect}(f/B) \ ,$$

which in the time domain will be

$$g_{b,B/2}(t) = B\text{sinc}(Bt) \tag{2.12}$$

and will be convoluted with the result of the previous point:

$$c_b(t) = \left[\left(e^{-j2\pi f_c(\cdot)}(c_p * g_{p,B})(\cdot)\right) * g_{b,B/2}\right](t) \ .$$

Baseband channel response

Since our signal $x(t)$ will be bandlimited, we can actually skip point 2 and get away with an equation such as

$$c_b(t) = [g_{B/2}(\cdot) * (c_p(\cdot)e^{-j2\pi f_c(\cdot)})](t) \ .$$

> In the equation we have a frequency dependent phase shift: we'll see that this is very nasty.

Pseudo-baseband channel response

We then have that

$$c(t) = c_p(t)e^{-j2\pi f_c t}$$

is the pseudo-baseband channel response, such that if $x(t)$ is bandlimited we can obtain

$$y(t) = [x(\cdot) * c(\cdot)](t)$$

and also such that

$$c_b(\tau) = [g_{B/2}(\cdot) * c(\cdot)](\tau) \ .$$

> This comes from my reasoning after these statements

The figure shows filtering with passband signals:

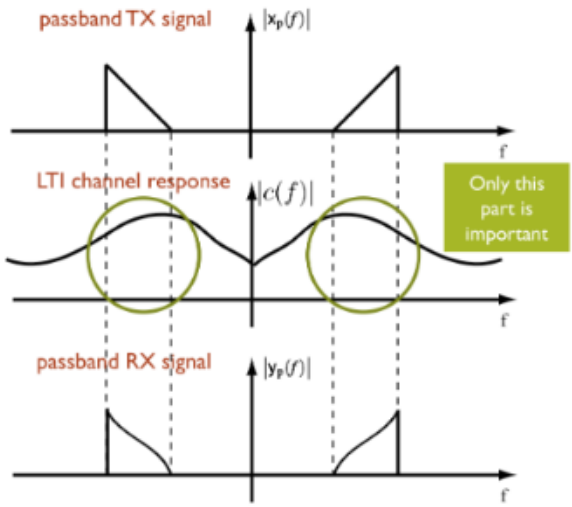
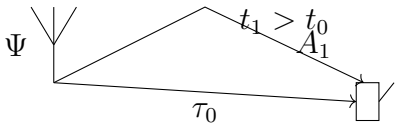


Figure 3.4: This image helps understanding why we simplify the equation of the channel response

3.9 Q paths complex baseband equivalent

Let's look at an example in which we have an antenna and a phone; we'll experience different delays τ_q and different gains A_q for each path followed by the signal:



We can model such a channel as

$$c_p(\tau) = \sum_{q=0}^{Q-1} A_q \delta(\tau - \tau_q) \; . \tag{2.20}$$

The baseband equivalent (according to "040299") will be

$$c_b(\tau) = g_{B/2}(\tau) * \sum_{q=0}^{Q-1} A_q \delta(\tau - \tau_q) e^{-j2\pi f_c \tau_q} \; . \tag{2.21}$$

which will give an output

$$\begin{aligned} y(t) &= [c_b * x](t) \\ &= \int_{-\infty}^{\infty} \left(g_{B/2}(\tau) * \sum_{q=0}^{Q-1} A_q \delta(\tau - \tau_q) e^{-j2\pi f_c \tau_q} \right) x(t-\tau)\, d\tau \qquad (2.22) \\ &= \int_{-\infty}^{\infty} \left(\sum_{q=0}^{Q-1} A_q \delta(\tau - \tau_q) e^{-j2\pi f_c \tau_q} \right) x(t-\tau)\, d\tau \qquad (2.23) \\ &= [c * x](\tau) \qquad (2.24) \end{aligned}$$

with $c(\tau)$ being the **complex pseudo-baseband channel response**, since it's shifted but not bandlimited. $*$ follows from x being bandlimited.

We can obtain the complex baseband channel response from the complex pseudo-baseband channel response as

$$c_b(\tau) = [g_{B/2} * c](\tau) \ . \tag{2.25}$$

3.10 Time discretization: discrete representation of the channel $c[\ell]$ (+critical sampling)

Since the channels are bandlimited, we can sample with a period $T \leq 1/B$ (equality is what we'll call *critical sampling*) and discretize the input and the channel.

Let's say we have $x(t), y(t)$ represented by their discrete-time samples with $T = 1/B$; we want to find a channel such that

$$y[n] = \sum_{\ell} c[\ell] x[n - \ell] \ . \tag{2.27}$$

FYI
This is a noiseless frequency selective, unitary gain channel model, as will be shown in (2.80).

If we try and discretize the convolution $[c_b * x]$ (with reference to (2.25) and previous equations) with a step $d\tau \to T$, we get

$$\begin{aligned} y(nT) &= (c_b * x)(nT) \\ &= \int_{-\infty}^{\infty} c_b(\tau) x(nT - \tau)\, d\tau && (2.28) \\ &\approx \sum_{\ell} c_b(\ell T) x(nT - \ell T) T && (2.29) \\ &= \sum_{\ell} T c_b(\ell T) x[n - \ell] \ , && (2.30) \end{aligned}$$

which, when compared with the equation of $y[n]$ in (2.27), will point us to conclude that $c[\ell] \approx T c_b(\ell T)$.

We can conclude that the discrete time equivalent in (2.27) for any baseband or pseudo-baseband channel is obtained by:

1. Taking the generic channel response $c(\tau)$
2. Filtering it with the low-pass filter $Tg_{B/2}(\tau)$, thus obtaining $c_b(\tau)$
3. Sampling it at $\tau = \ell T$

Discrete time baseband channel equivalent
It will thus be

$$c[\ell] = [Tg_{B/2} * c](\tau)|_{\tau=\ell T}$$

or, if the channel is already bandlimited,

$$c[\ell] = [Tc(\tau)]|_{\tau=\ell T} \,. \tag{2.31}$$

Where the T is introduced in order to keep the energy constant.
If we want to deal with signal theory, professor suggests to look at "unified signal theory".

What's important here is that we'll forget about the continuous time domain and we continue with the discrete time domain:

08.2_pp_57_130_A_signal_processing_perspective, page 10
In light of the connection established between passband signals, bandlimited baseband signals, and discrete-time signals, most results in this book are formulated directly in discrete time. Although actual signals are not exactly bandlimited since they are time-limited, and the filters in the front-ends are never ideal, signals are close to bandlimited as they are mandated to have low adjacent channel interference. Commercial systems also feature analog-to-digital and digital-to-analog converters with finite precision. Normally, perfect conversion is assumed in the design stages and then the required tolerances are investigated after the fact via simulation. Quantization and reconstruction errors are not part of the models in this text.

3.11 Idea of pulse shaping: from a digital signal $x[n]$ to an analog signal

Pulse shaping: the book mentions it, but it's not really important.

08.2_pp_57_130_A_signal_processing_perspective, page 10
The sampling and reconstruction that we have applied to discretize the transmit-receive relationships can be interpreted as basic modulation and demodulation schemes, whereby the complex symbols that make up the codewords are embedded onto $x(t)$ and then extracted from $y(t)$.

The basic concept of pulse shaping is to link the symbols of the message we're transmitting, $x[n]$, with the actual continuous time signal, $x(t)$.

If we know $x[n]$, we might think ab using sinc for interpolation; problem is that it is not causal! What we generally do then is to use the **square root raised cosine**.

No matter what functions we'll use for interpolation at the transmitter, we'll call it g_{tx}. At the receiver we'll use a different function, g_{rx}, to filter what we received.

We can generally **reconstruct the signal through interpolation as**

$$x(t) = \sum_n x[n] g_{tx}(t - nT) \ , \tag{2.41}$$

where $g_{tx}(\cdot)$ is the transmit pulse shape.

At the receiver, we'll then use a specific filter which in reality cannot be the ideal low-pass filter $g_{B/2}(\cdot)$ described in (2.18): it will be a modified filter $g_{rx}(-)$.

The **composite pulse shape** is thus defined by combining the two pulse shapes, as

$$g(\tau) = [g_{tx} * g_{rx}](\tau) \ . \tag{2.42}$$

Skipped
Possible definitions of $g(\tau)$ can be found from 08.2_pp_57_130_A_signal_processing_perspective, page 13 to 08.2_pp_57_130_A_signal_processing_perspective, page 15. Professor completely skipped them.

Week 17 Nov - 23 Nov

3.12 Channel models (frequency selective, FIR, flat channel); channel normalization and average symbol energy; vector representation of a SISO FIR channel

An important thing is the normalization of the channel; people normally make lots of mistakes about this.

We have the model defined in (2.19). We can model such a channel as

$$c_p(\tau) = \sum_{q=0}^{Q-1} A_q \delta(\tau - \tau_q) \ . \tag{2.19}$$

If we don't pay attention, this channel could be amplified. We need to pay attention because the energy of the channel is

$$\sum |A_q|^2 \ ,$$

which might be greater than 1.

> What's important is the ratio between the noise and the amplification of the channel (which comes from the antenna itself, its directionality etc etc): the gain is in respect to an isotropic antenna.

Chapter 4

Week 17-23 November

4.1 Channel models (frequency selective, FIR, flat channel); channel normalization and average symbol energy; vector representation of a SISO FIR channel

An important thing is the normalization of the channel; people normally make lots of mistakes about this.

We have the model defined in (2.19). We can model such a channel as

$$c_p(\tau) = \sum_{q=0}^{Q-1} A_q \delta(\tau - \tau_q) \ . \tag{2.19}$$

If we don't pay attention, this channel could be amplified. We need to pay attention because the energy of the channel is

$$\sum |A_q|^2 \ ,$$

which might be greater than 1.

> What's important is the ratio between the noise and the amplification of the channel (which comes from the antenna itself, its directionality etc etc): the gain is in respect to an isotropic antenna.

In short, the problem is that we want to separate the gain from the channel response, which is why we write:

$$c(\tau) = \sqrt{G} h(\tau) \tag{2.64}$$

$$c_b(\tau) = \sqrt{G} h_b(\tau) \tag{2.65}$$

$$c[l] = \sqrt{G} h[l] \tag{2.66}$$

we'll also impose that:

$$\sum_l \mathbb{E}|h[l]|^2 = 1 \tag{4.1}$$

the channel gain should always be one.

Rem: This makes perfect sense. *We know that a generic request on the channel is that*

$$\sum_l \mathbb{E}\|\boldsymbol{H}[l]\|_F^2 = N_t N_r$$

here we're in SISO, thus we have only one receiver and one transmitter; it makes perfect sense to request that the sum of the expectations is 1.

Def 4.1.1: Frequency selective model. *The **frequency selective** is a model in which we have $\sqrt{G}$ then a causal and FIR (Finite Impulsive Response) channel and then noise which is IID and $v[n] \sim \mathcal{N}_{\mathbb{C}}(0, N_0)$.*

The output of a generic channel, assuming the normalization we mentioned holds, would be:

$$y[n] = \sqrt{G}\sum_l h[l]x[n-l] + v[n] \tag{2.69}$$

If the condition of FIR and of causality hold, though, the output can be simplified as:

$$y[n] = \sqrt{G}\sum_{l=0}^{L} h[l]x[n-l] + v[n] \tag{2.70}$$

If we then have $L = 0$, the channel is multiplicative and will be called **flat channel**, giving an output of:

$$y[n] = \sqrt{G}h[0]x[n] + v[n] \tag{2.71}$$

all the frequencies are treated the same; we can have this model represent a time variant setup if $h[0]$ actually depends on n.

If $L > 0$ the channel exhibits ISI (Inter-Symbol Interference) and since its response will not be flat in frequency, we say that it is **frequency-selective**.

Symbols are normalized to have an **average symbol energy**:

$$E_s = \frac{1}{N}\sum_{k=0}^{N-1} \mathbb{E}|x[k]|^2 \tag{4.2}$$

We'll see that this specific definition is just the per-block power constraint. We can also have per-symbol or per-antenna power constraints, from which will follow other constraints on the precoding matrix (see signal power constraints).

This will allow us to write the SNR as:

$$\text{SNR} = \frac{P_r}{N_0/T} = \frac{GE_s}{N_0}, \tag{4.3}$$

where:

- P_r is the power of the received signal, which in turn is equal to $P_r = GP_x = GE_s/T$ (P_x is the power of the transmitted signal);
- N_0 is the covariance of the AWGN.

This last relation will turn out to be useful later on (it is literally (4.2), (4.3) and it will be of use to write (4.6), (4.7)...).

If we have a frequency-selective channel, we can give a vector representation of (2.70). For the channel we'll use a Toeplitz matrix, which is a $N \times (N+L)$ convolution matrix defined as:

$$\boldsymbol{H}_{N,N+L} = \begin{bmatrix} h[L] & \dots & h[0] & 0 & \dots & 0 \\ 0 & h[L] & \dots & h[0] & 0 & \vdots \\ \vdots & & \ddots & & \ddots & 0 \\ 0 & \dots & 0 & h[L] & \dots & h[0] \end{bmatrix} \tag{2.72}$$

the input signal is consequently defined as:

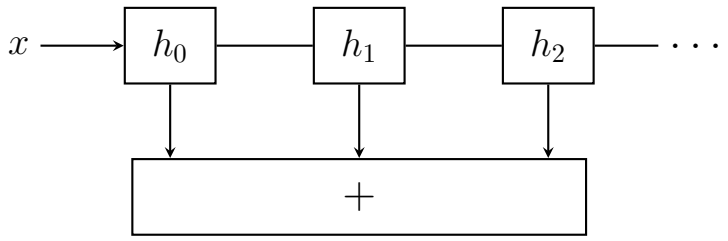
$$\boldsymbol{x}_{N+L} = \begin{bmatrix} x[-L] \\ \vdots \\ x[-1] \\ x[0] \\ x[1] \\ \vdots \\ x[N-1] \end{bmatrix} \tag{2.73}$$

giving that if the channel noise is $\boldsymbol{v}_N$, we'll have:

$$\boldsymbol{y}_N = [y[0] \dots y[N-1]]^T = \sqrt{G}\boldsymbol{H}_{N,N+L}\boldsymbol{x}_{N+L} + \boldsymbol{v}_N \tag{2.74}$$

We can show that whichever n-th row in (2.74) is equivalent to (2.70).

This is like if we had a MIMO system: "Vectorized result makes an equivalence between frequency-selective SISO channels and frequency-flat MIMO channel, and may be useful for deriving estimators or equalizers".



Block diagram of the FIR/Channel model (reconstruction)

This is the same of having a MIMO channel with different gains. We can ask: if we want to estimate a symbol, how can we only get the symbol $x[n]$ we're interested in from $y[n]$?

We'll want to massage the MIMO system to make it similar to our model; we can put together all the matrices and get one only matrix: this is the stacked vector representation.

4.2 Organization detour (+mentions of SVD, Precoding)

Organizational issue: he set up a lecture for next wednesday; this is because on the 1st of November we'll have no lecture. Professor tried to keep the target so that we could finish before christmas. We'll also have some other lectures to look over.

One of the next topics is OFDM; he suggests to read the part of the book dealing with it before going through it with lectures; if we read it beforehand, we'll be in a better position.

Lore of the course moment About MMSE: when we transmit, this is another way to estimate the symbol transmitted.

Blabbering ab MIMO: if we have N tx and M rx, we have NM paths.

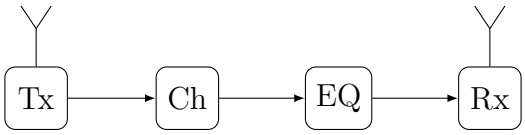
We'll see filters whose coefficients are matrices.

Singular value decomposition: we'll have a specific slide on this.

Precoding: we should try, if possible, to figure what a certain equation means in practice; the antenna may have a main lobe and we can change the main lobe by moving the antenna mechanically. Precoding is changing the type of transmission without moving the antenna mechanically; we do this using a certain matrix which we call precoding. With precoding we can steer where the antennas are transmitting.

We'll move to OFDM (used for 4G, 5G and probably will also be used for 6G); we need to see why we use OFDM! That's the wrong approach, since there are other types of modulation (single carrier modulation, just as an example). When we're transmitting, we have two antennas and a single channel.

We model the channel via spans:



If we have reflection, the previous signal might superpose to the previous one. The channel has a non-linear response: we can equalize it! We have many ways to equalize the channel!

The EQ in MIMO would increase the noise! We thus look at two **different types of equalizations**; we can have a linear or a non-linear equalization, but the latter is more resource-heavy.

We'll then look at the basics of radio propagation and such; there will be many concepts which are really important and will be pointed out. We'll see the Friis equation.

◇ **Exam**

Example of exam question: we can transmit at 900MHz or at 24GHz; we have a fixed power P and we can decide which frequency.
Thanks to the Friis equation, we know that choosing the lower frequency is better.

The state is getting more money from choosing the higher frequency.

4.3 Problems in discretization: multiple paths, multiple taps (+synchronization, reflection and noise whitening)

We've seen that long time ago people were manipulating analog signals, aka continuous time. Microphones are still in frequency modulation, as an example. Long time ago, we were able to communicate and we supposedly went to the moon.

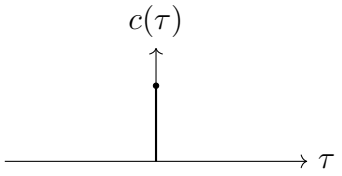
Kalman filtering learned about his filter in Moscow.

We were used at working with analog signals; the leap happened when we began manipulating signals with computers: in order to do this, we need to discretize signals. Analog circuits are different from one another; computers always do the same operations! It's much better for us to manipulate signals in a discrete domain!

What if we have a certain channel with a response which is an impulse?

4.4 Discretized Channel Model

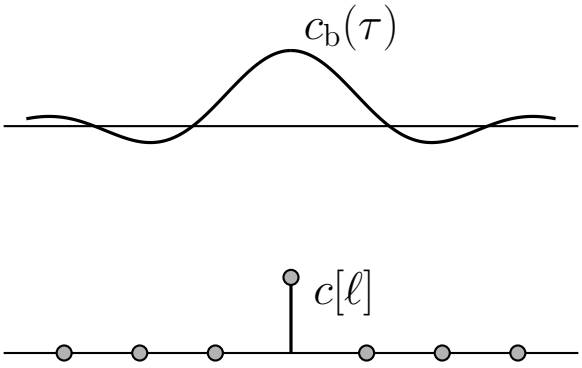
One-tap



Whatever is transmitted, it arrives straight to the receiver. This is true up to a certain point: it might take a while for the signal to travel up to the destination: we might want to move our delta! This is a problem which is called **time synchronization**.

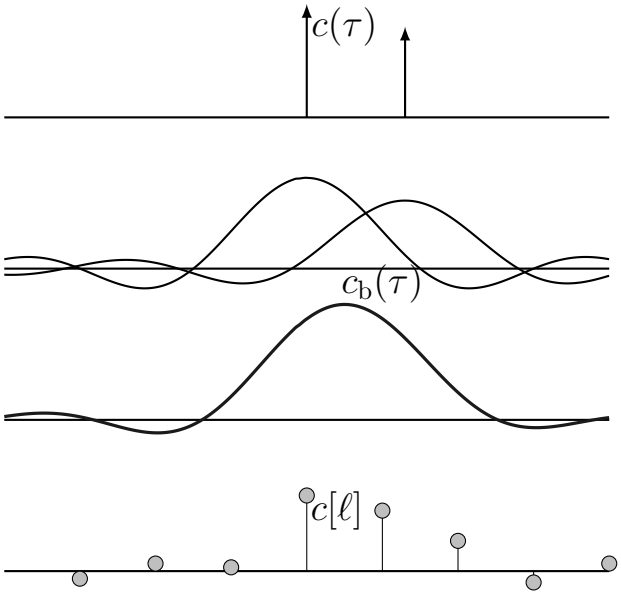
How can the transmitter understand when $t = 0$ is? We might have a form of mismatching if the receiver doesn't understand correctly when the signal started. We might do syncing using autocorrelation: CAZAC sequences are generally used (Constant Amplitude Zero AutoCorrelation waveform).

Here we suppose that our signal is synchronized; let's convolve it with a sinc; we have that sampling it will give a Kronecker delta function.



This only happens if we have synchronization.

Let's look at another channel: we have two paths which will be followed by our signals;



This is a typical example of **reflection**; for each delta, we'll have a sinc; if we sample this, then we'll have four taps (as can be seen in the last figure). In this case what happens is that from two taps, in the discrete domain we got 4.

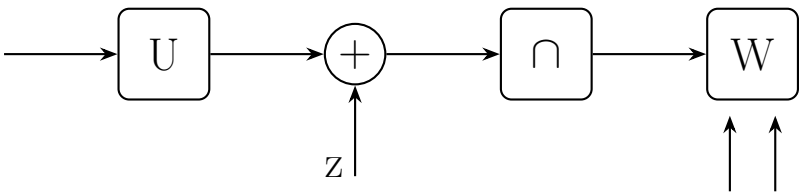
The amount of taps are increasing; we can only sync with one or the other, thus we're always in trouble.

This is a simple problem: if we only have a tap, we'll have a single tap in the frequency domain. If we have multiple taps, stuff might become a nightmare!

Sampling is nice, but we can't do this for free.

Let's make some comments on developing a discrete time model:

- Noise is still IID and the noise samples are still Gaussian with $\sim \mathcal{N}(0, N_0)$
- We can also use root-raised-cosine filter instead of a sinc filter.
- If we want to detect a signal, it's much easier to do so if we have a white specter: if the noise goes through certain blocks, though, it will no longer be flat!

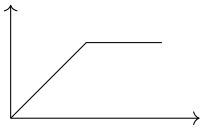


we can add a **block which whitens the noise**; this is good since then all the subsequent equalization and such work much better. Whitening a noise can be much more difficult.

Pulse shaping: if in the frequency domain we have a sinc, this is optimal. This is a sinc tho, which decays slowly in the time domain. This is why we're using a root-raised-cosine filter: it will have a wider bandwidth but it will be better in the time domain.

Instead of sampling at the Nyquist rate (which allows for perfect recovering), we can say: "what if I sample the signal more frequently? Maybe I can make algorithms which

are working better". From the Information theory FOV, Nyquist suffices; but if we have to do with practical issues like non linearities, stuff might be different. Amplifiers are generally non-constant: there is a certain range in which the amplifier is linear, but there is a certain point at which whichever signal we put at the input of the amplifier, we'll get the exact same output; this is called clipping:



For synchronization, it helps to have oversampling (not in the book!); if the Nyquist rate is just in between the right points, it might not sync correctly.

Random stuff about vinils

4.5 Transitioning to MIMO (frequency selective and FIR channel expressions; channel normalization)

We're now going towards MIMO; we have a certain number N_t of transmitting antennas and N_r of receive antennas; those numbers can def be different.

We have at the input the value $\mathbf{x}$ and at the output $\mathbf{y}$ with noise $\mathbf{v}$.

Let's start from the causal and FIR **frequency flat** channel; we'll have an output of:

$$\mathbf{y}[n] = \sqrt{\rho}\mathbf{H}\mathbf{x}[n] + \mathbf{v}[n]. \tag{2.81}$$

$\mathbf{H}$ is a matrix such that the impulse response between transmit antenna j and receive antenna i will be represented by $\mathbf{H}_{i,j}$.

◇ Frequency flat (also found as frequency-flat)

By frequency flat, the book refers to a channel which has just one tap, aka $L = 0$.

$\mathbf{x}$ is multiplied by $\mathbf{H}$; this means that the **length of $\mathbf{x}$ is** N_t; the length of $\mathbf{y}$ is N_r.

Skipped

Professor skipped the beginning of **section 2.3.1**, which tackles a continuous IO MIMO relation.

Just like we saw, we're not certain of having a delta; we might have some other paths! We'll model this (a **FIR channel**) just as we did before:

$$\mathbf{y}[n] = \sqrt{\rho}\sum_{l=0}^{L-1}\mathbf{H}[l]\mathbf{x}[n-l] + \mathbf{v}[n]. \tag{2.80}$$

where $[\mathbf{H}[l]]_{i,j}$ represents the response between transmit antenna j and receive antenna i. This means that the **output vector at time n is also depends on previous symbols**. Each vector is then also weighted by a different matrix, which changes over time.

There is no difference, conceptually, with the scalar case; we only have that we have a matrix instead of a vector.

Before, we had normalized the channel; in this case we sum over all the three indexes; we don't want it to be 1, but instead we'd like to be $1 \cdot \#$ of paths $= \boxed{N_t N_r}$.

We can use the definition of the **Frobenius norm** in order to impose the normalization; while we could impose the summation to be equal to 1, this would not allow to include channel models with power asymmetry across antennas; for this reason, we have that

$$\sum_{l=0}^{L-1} \sum_{i=0}^{N_r-1} \sum_{j=0}^{N_t-1} \mathbb{E}[|H[l]_{i,j}|^2] = N_t N_r. \tag{2.82}$$

which can be rewritten more compactly using the definition of Frobenius norm (given in appendix B.5),

$$\begin{aligned} \sum_{l=0}^{L-1} \mathbb{E}[||\mathbf{H}[l]||_F^2] &= \sum_{l=0}^{L-1} \mathbb{E}[\mathrm{tr}(\mathbf{H}[l]\mathbf{H}^H[l])] && (2.83) \\ &= \sum_{l=0}^{L-1} \mathbb{E}[\mathrm{tr}(\mathbf{H}^H[l]\mathbf{H}[l])] && (2.84) \\ &= N_t N_r. && (2.85) \end{aligned}$$

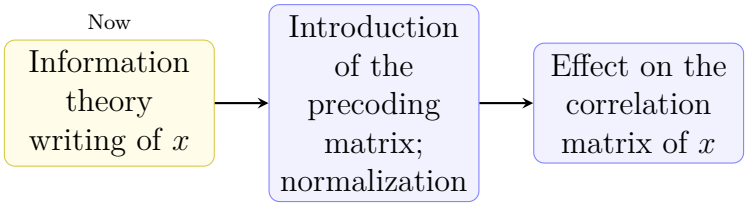
which for the frequency-flat channel simplifies to

$$\begin{aligned} \mathbb{E}[||\mathbf{H}||_F^2] &= \mathbb{E}[\mathrm{tr}(\mathbf{H}\mathbf{H}^H)] \\ &= \mathbb{E}[\mathrm{tr}(\mathbf{H}^H\mathbf{H})] \\ &= N_t N_r. \end{aligned} \tag{2.86}$$

The Frobenius norm is taking matrix and substituting it with the trace of the matrix multiplied with the complex conjugate of said matrix.

4.6 Precoding

4.6.1 Introduction (information theory writing of $x[n]$, matrix F; variation of precoded x)



We have the precoding; we'll start with SISO signals. We'll decompose it in an IT (Information Theory) friendly way:

$$x[n] = \sqrt{E_s P[n]} s[n].$$

where:

- E_s is the average symbol energy

- $P[n]$ is the power allocation at a certain given time
- $s[n]$ are unit-variance symbols (PSK, QAM, Gaussian)

Hello, new writing of $x[n]$! Will you stick around?

Yup. This writing will be reused in chapter 4, when linking OFDM and vector coding (and also when finding the SNR for both).

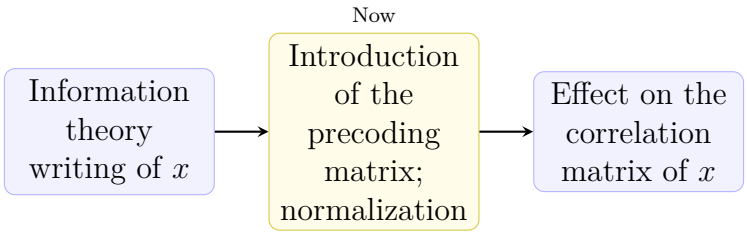
About units...

From what I gather, if we admit that energy is in $W \cdot s$ and power in W, if x is in units of V, then s must have a unit of $V/\sqrt{J}$.

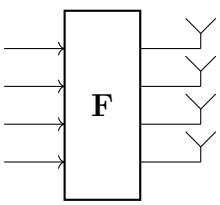
Sending info using a Gaussian: a **Gaussian** is the base we use for the other ways to send signals; we use it as a **bound** for what we can do. If we don't reach the performance of the Gaussian, there is a reason for that.

When we're going from 16 to 64 QAM, we're just making the constellation more dense.

$P[n]$ is the power allocation for each signal; about it:
In our phone we have an algorithm called the *power control algorithm*; when we go away from an antenna, we need to raise the power used to transmit; if we're close, we need to have a forward signal!
What is practically done is that we measure the signal from the base station and we'll know how strong our signal should be.
We'll then change the signal when we go farther from a station.



With MIMO we do *transmit precoding*: we suppose we have $N_s \leq N_t$: the number of transmit symbols (N_s) is at most equal to the number of transmit antennas (N_t). If we're not using an antenna, we can think about what to do with it. If we have 5 symbols but just an antenna, we won't be able to transmit that symbol. **Our vector of symbols can only be transmitted if** $N_s \leq N_t$. In the MIMO we can do more than deciding how much power we should use.



we might decide to use a matrix to mix those symbols; it's a degree of freedom which might prove very useful: why not use it?

For any running symbol, we multiply it with a matrix $\mathbf{F}[n]$ (which can vary with n!!):

$$\mathbf{x}[n] = \sqrt{\frac{E_s}{N_t}}\mathbf{F}[n]\mathbf{s}[n]. \tag{2.96}$$

where:

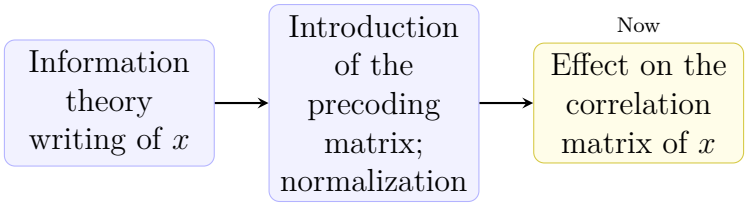
- The $s[n]$ symbols are **IID**.
- E_s is the average energy per symbol (more about this later).
- $F[n]$ is the precoder matrix; as we'll see, it's normalized in order to respect specific relations. It will have a dimension of $N_t \times N_s$.

We have a restriction on $F[n]$ which uses the **Frobenius norm**; it grants that the antenna is not amplifying the signal:

$$\frac{1}{N}\sum_{n=0}^{N-1}||\mathbf{F}[n]||_F^2 = \frac{1}{N}\sum_{n=0}^{N-1}\text{tr}(\mathbf{F}[n]\mathbf{F}^H[n]) = N_t. \tag{2.97}$$

Later, we'll need the transmit covariance. $s[n]$ are IID codewords, which means that $\mathbb{E}[\mathbf{s}[n]\mathbf{s}^*[n]] = \mathbf{I}$ (this follows from the fact that independence implies uncorrelation and that these vectors are 0-mean; the main diagonal will have the variance of each signal, which is 1), with dimensions $N_s \times N_s$.

We're always transmitting 0-mean and 1-variance signals.



If we take into account $x[n]$, which has multiplication with the matrix, the covariance will be given by $\mathbf{R_x}$. In order to find its value, we'll apply the fact that $(\mathbf{Fs})^* = \mathbf{s}^*\mathbf{F}^*$. We'll thus have

$$\begin{aligned}\mathbf{R_x} &= \frac{E_s}{N_t}\mathbb{E}[\mathbf{F}\mathbf{s}\mathbf{s}^*\mathbf{F}^*] \\ &= \frac{E_s}{N_t}\mathbf{F}\mathbb{E}[\mathbf{s}\mathbf{s}^*]\mathbf{F}^* \\ &\overset{(*)}{=} \frac{E_s}{N_t}\mathbf{F}\mathbf{F}^*.\end{aligned}$$

re-introducing the time dependent notation, we have (for ID $\mathbf{s}$)

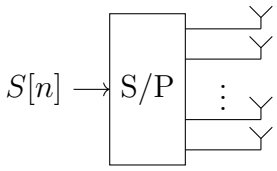
$$\mathbf{R_x}[n] = \frac{E_s}{N_t}\mathbf{F}[n]\mathbf{F}^H[n]. \tag{2.101}$$

Comparison with normal case

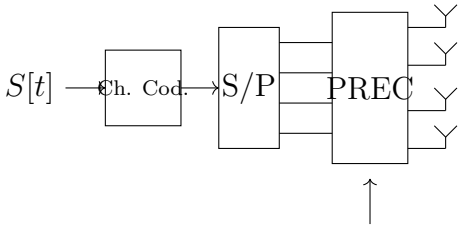
If we were to set $\mathbf{F}[n] = \mathbf{I}$, we'd find that the non-precoded matrix of x is

$$\frac{E_s}{N_t}\mathbf{I}.$$

We have a source of symbols, we pass them through a serial to parallel and then we go through the antennas; usually, we use a code



What we generally do is actually code after the series/parallel transition!



4.7 Recap of singular value decomposition

Let's revisit the concept of singular value decomposition!

This is heavily used in machine learning and such

We have a certain matrix $\mathbf{A}$ of dimensions $N \times M$; we can always decompose it as

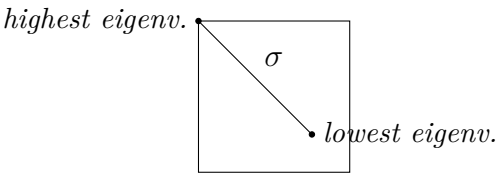
$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^*,$$

where:

- $\mathbf{U}$ is unitary $N \times N$.
- $\mathbf{V}^*$ is unitary $M \times M$.
- $\mathbf{\Sigma}$ is $N \times M$ with non negative entries on its diagonal ordered from largest to smallest.

The entries of said matrix **are the singular values of the original matrix, aka the square root of the eigenvalues of A**.

The $\mathbf{\Sigma}$ matrix is generally like



If we choose $\mathbf{u}_j$, a column of $\mathbf{V}$ and $\mathbf{u}_j$, a column of $\mathbf{U}$, we'll have

$$\mathbf{A}\mathbf{v}_j = \sigma_j \mathbf{u}_j$$
$$\mathbf{A}^*\mathbf{u}_j = \sigma_j \mathbf{v}_j$$

where σ_j is the j-th element on the diagonal of $\mathbf{\Sigma}$.

We just need to remember the dimensions in order to have this work correctly.

Property of unitary matrices

For a unitary matrix, we have that

$$\mathbf{U}^{-1} = \mathbf{U}^*.$$

This is why unitary matrices are very useful.

$\mathbf{U}$ is very easy to invert and same goes for $\mathbf{V}$. Then we might say: it will be easy to invert $\mathbf{A}$.

$\mathbf{A}$ will also have the following properties:

Table B.1 Bases for the column, row, and null spaces of an $N \times M$ matrix $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^*$

Subspace	Dimension	Basis
Column space of $\mathbf{A}$	$r = \text{rank}(\mathbf{A})$	First r columns of $\mathbf{U}$
Row space of $\mathbf{A}$	r	First r columns of $\mathbf{V}$
Null space of $\mathbf{A}^T$	$(N - r)$	Last $(N - r)$ columns of $\mathbf{U}$
Null space of $\mathbf{A}$	$(M - r)$	Last $(M - r)$ columns of $\mathbf{V}$

About the number of streams

If we have 1 transmitter and 1 receiver, we know many things; but we didn't consider a channel which is dispersive and generates ISI; how do we deal with ISI?? This is handled in section 2.5 of the book.

Let's suppose, at least for now, that we have no ISI; the Shannon bound tells us that we have a maximum amount of bits per symbol per hertz which we can put through the channel.

People said: we want to transmit even more!! In order to do so, we increase the number of transmitter and receiver antennas. Let's say we have 3 transmitters and 5 receivers; the interactions between all of these won't be described by a vector anymore, but rather by a matrix. We'll see that we have **a number of independent streams which is**

$$\#\text{streams} \leq \min\{N_t, N_r\};$$

we use MIMO since we want to be able to transmit even more than what would have been possible while respecting the Shannon bound. It's useless to have 1 transmitter and many receivers: we'd be able to only have 1 stream.

It's difficult to have many antennas in the phone, much more than it is to have many in our station.

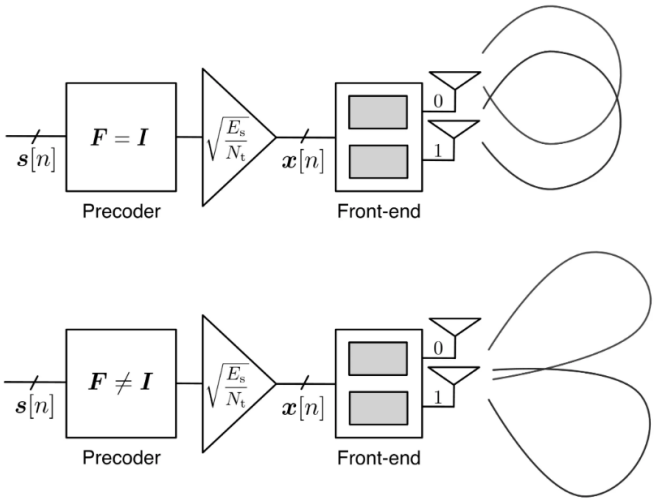
Before looking at this, we want to understand if this works at all, though! We first need to know the theory and that this is possible and only then will we be able to use this.

4.8 Applying precoding: decomposing F and explaining the various parts of its new form

Let's apply what we've learned to the model

$$\mathbf{x}[n] = \sqrt{\frac{E_s}{N_t}}\mathbf{F}[n]\mathbf{s}[n], \tag{2.98}$$

we know we have $\mathbf{s}[n]$ at the input and let's suppose that $\mathbf{F} = \mathbf{I}$: we go directly in the antennas. If we do this, all the antennas transmit all like a circle (this depends on how they're built), as it comes. If we then have $\mathbf{F} \neq \mathbf{I}$, we can get each antenna to point to a specific user; this allows us to maximize the gain we have from the transmit antenna.



What we want to do is to multiply our symbols with a matrix and do something. What can we achieve? In order to better understand this, we can use singular value decomposition to decompose $\mathbf{F}$ as:

$$\mathbf{F}[n] = \mathbf{U}_F[n]\mathbf{\Sigma}_F[n]\mathbf{V}_F^H[n], \tag{2.100}$$

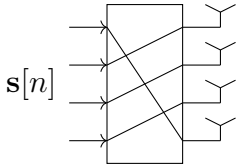
where:

- $\mathbf{V}$ is the **mixing matrix**, which is a $N_s \times N_s$ unitary matrix
- $\mathbf{\Sigma}$ is the **power allocation matrix** and is $N_t \times N_s$ (more about this later)
- $\mathbf{U}$ is the **beam steering matrix**, which is a $N_t \times N_t$ unitary matrix

If we had only

$$\mathbf{V}_F^*\mathbf{s},$$

we'd get N_t symbols; this mixes the various signals towards the various antennas:



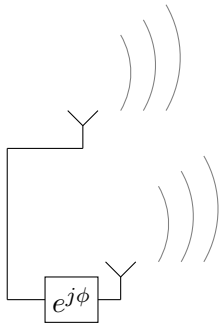
By only doing this we'd create confusion; this is why we also have a power allocation matrix: it has a diagonal which expresses how much power we'll give to each antenna. It will have a shape like

$$\mathbf{\Sigma}_F[n] = \begin{bmatrix} \mathbf{P}^{\frac{1}{2}} \\ \mathbf{0} \end{bmatrix} \tag{2.101}$$

where $\mathbf{P}$ is a $N_s \times N_s$ matrix which will be defined below.

What's really important is $\mathbf{U}_F$, which allows us to **steer the signal**.

This had been discovered a long time ago! If we have a radar and we transmit with a phase shift on one antenna:



In a certain direction we'll have a specific ϕ; if $\phi = \pi$, we're not transmitting anything. If we look at another direction, we can tilt an antenna electronically to transmit over a certain direction.

This is used by radars, even though radars transmit an impulse and want to get stuff back. In order to understand where the signal is coming from they use multiple antennas and they use something like a steering matrix. If we take the nose of an airplane, the whole nose of an airplane has a radar in it.

It's nothing new in this sense!

So now we can have different directions from an array of antennas. By using appropriate matrices, we can make spatial streams.

Our tools for designing are $\mathbf{U}_F$ and the amount of power we put in each antenna (the sum of the power cannot be more than N_t), expressed by the matrix

$$\mathbf{P} = \text{diag}(P_1, \dots, P_{N_s-1})$$

such that $\sum_{i=0}^{N_s-1} P_i = N_t$.

The notation P_j is especially interesting since the jth mixed signal (aka $[\mathbf{V}_F^H \mathbf{s}[n]]_j$) will have a power of $\frac{E_s}{N_t} P_j$ allocated to it, where the E_s/N_t term comes from our definition in (2.98).

The jth amplified signal will be launched from all transmit antennas and will be weighted with the coefficients in the jth column of $\mathbf{U}_F[n]$.

If we were to choose a silly (aka useless) setup, we might have both $\mathbf{U}_F, \mathbf{V}_F$ be identity matrices and $\mathbf{P} = \frac{N_t}{N_s}\mathbf{I}$, which entails

$$\mathbf{F}[n] = \sqrt{\frac{N_t}{N_s}} \begin{bmatrix} \mathbf{I}_{N_s} \\ \mathbf{0} \end{bmatrix}. \tag{2.102}$$

With such trivial precoder, each of N_s antennas directly radiates one of the data streams within $\mathbf{s}[n]$ while the remaining $N_t - N_s$ antennas are silent; the transmit signals are independent. If this is the case, the transmission is said to be *isotropic* from a precoding pov or even *unprecoded*.

Example 2.10

Let $N_t = N_s = 2$ and suppose that the two signals within $s[n]$ are QPSK-distributed. Examine the structure of the transmit signal $x[n]$ produced by a time-invariant precoder.

Solution

Dropping the dependence on n, let $\mathbf{P} = \text{diag}(P_1, P_2)$ and

$$\mathbf{U}_F = \begin{bmatrix} U_{00} & U_{01} \\ U_{10} & U_{11} \end{bmatrix}, \qquad \mathbf{V}_F = \begin{bmatrix} V_{00} & V_{01} \\ V_{10} & V_{11} \end{bmatrix}. \tag{2.103}$$

The application of the mixing matrix to $\mathbf{s}$ yields the mixed signal vector

$$\mathbf{V}_F^* \mathbf{s} = \begin{bmatrix} V_{00}^*(\mathbf{s})_0 + V_{10}^*(\mathbf{s})_1 \\ V_{01}^*(\mathbf{s})_0 + V_{11}^*(\mathbf{s})_1 \end{bmatrix} \tag{2.104}$$

whose unit-variance entries conform to 16-ary distributions, in general distinct. Then, after power allocation,

$$\sqrt{\frac{E_s}{2}} \mathbf{P}^{1/2} \mathbf{V}_F^* \mathbf{s} = \begin{bmatrix} \sqrt{\frac{E_s P_1}{2}} (V_{00}^*(\mathbf{s})_0 + V_{10}^*(\mathbf{s})_1) \\ \sqrt{\frac{E_s P_2}{2}} (V_{01}^*(\mathbf{s})_0 + V_{11}^*(\mathbf{s})_1) \end{bmatrix}. \tag{2.105}$$

The top signal is launched from the two antennas, weighted by U_{00} and U_{10}, while the bottom signal is launched weighted by U_{01} and U_{11}. If $\mathbf{F} = \mathbf{I}$, then each signal is radiated from only one antenna. Conversely, if $\mathbf{F} \neq \mathbf{I}$, each signal is transmitted from both antennas and steered in a certain direction. Both possibilities are illustrated in Fig. 2.11.

Comments:

- The $*$ operator on matrix also implies transposition
- A unitary matrix is just rotating the vector: it won't change its magnitude
- If $\mathbf{F} = \mathbf{I}$, each signal is radiated by one antenna
- If $\mathbf{F} \neq \mathbf{I}$, both signals are transmitted by both antennas and we can steer the antennas.

4.9 Signal power constraints

Which power constraint will we enforce? We might have average energy per block or per symbol or even per antenna:

- **Per-block** $\frac{1}{N} \sum_n ||\mathbf{x}[n]||^2 \leq P$, or equivalently $\frac{1}{N} \sum_n \text{tr}(\mathbf{R}_\mathbf{x}[n]) = P_t$.

  → Then the precoder $\frac{1}{N} \sum_n ||\mathbf{F}[n]||_F^2 = N_t$

- **Per-symbol** $\mathbb{E}[||\mathbf{x}[n]||^2] = E_s$ or $\text{tr}(\mathbf{R}_\mathbf{x}[n]) = E_s$

  → Then the precoder satisfies $\text{tr}(\mathbf{F}[n]\mathbf{F}^H[n]) = \text{tr}(\mathbf{P}[n]) = N_t$

- **Per-antenna** $(\mathbf{R_x})_{ii} = \frac{E_s}{N_t}$, which means that

  $||\mathbf{F}[n]||^2_{i,:} = 1$ for $i = 0, \dots, N_t - 1$

We might even be super strict and say *every symbol should have the same energy*, or more relaxed and say: on average a block should have a certain amount of energy. What we have depends on our situation.

Examples

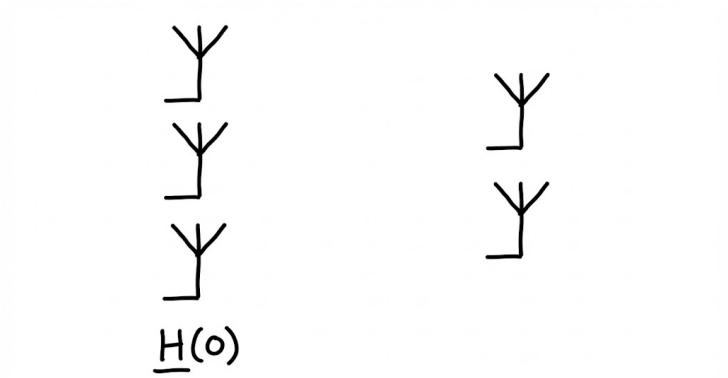
Regulation says that if we put a sphere around the top of the antenna; the power of the outgoing em field should be below a certain value. If we have that power, we'll need to share it between antennas.
If we have DAS (Distributed Antenna System), which is now used in tunnels, we can put the same power on the various antennas.

The constraint depends on the practical situation which we have.

4.10 Vectorizing the channel: MIMO multipath channel (specifically, stacked notation absorbing convolution, time dependency and MIMO) for receiver i and for the whole system

We were looking at how to vectorize the channel; this paragraph, specifically, investigates how to reach for MIMO a result similar to the (2.74) we reached for SISO. We also include previous values of the signal since the channel has memory. This is like convolution, but in the time domain.

Now, instead of having SISO systems, we can jump to the MIMO case. In summary, instead of having scalars in the matrix H, in the MIMO system we'll have matrices.



$H(0)$ tells how the signal gets from one part to the other. In the SISO case we had $h[k], k = 0, \dots, L$: we had L paths since we weren't in a flat channel. We won't stop at considering only the matrix corresponding to $h[0]$; we'll also look at the matrix for the system's memory.

If we then consider the receiver antenna i, we'll have that the received signal will be the

sum of all the values:

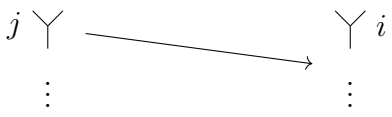
$$y_i^{(r)} = \sqrt{\rho} \sum_{j=0}^{N_t-1} [\mathbf{H}]_{i,j} \mathbf{x}_j^{(N+L)} + \mathbf{v}_i^{(r)}. \tag{2.89}$$

where $\mathbf{H}$ is a matrix of matrices and $N+L$ is there since we have L delays. The output does not only depend on one input vector, but it's still a vector of dimension $N+L$ depending on the input of each transmit antenna ($x_j^{(N+L)}$).

What are those $N, N+L$ subscripts?

Those subscripts represent the dimensions of vectors and matrices. Just as an example, $\mathbf{x}$ is a $(N+L) \times 1$ vector and $\mathbf{H}$ is a $N \times (N+L)$ matrix.
Specifically, then, each vector contains the time information for signals for $n = 0, \dots, N-1$, with the relation $\mathbf{Hx}$ representing the convolution operation.

We're considering the antenna i:



SISO!

We have many antennas transmitting to i; we need to sum over all the transmitting antennas and over all the vectors transmitted at the j-th antenna. The jump we're doing with this new equation is given by the fact that we have many transmission antennas! y will have a dimension of $N_r N$! We're putting the various vectors one on top of the other:

$$\bar{\boldsymbol{y}}_{N_r N} = \begin{bmatrix} \bar{\boldsymbol{y}}_N^{(0)} \\ \vdots \\ \bar{\boldsymbol{y}}_N^{(N_r-1)} \end{bmatrix}.$$

$$\bar{\boldsymbol{x}}_{N_t(N+L)} = \begin{bmatrix} \bar{\boldsymbol{x}}_{N+L}^{(0)} \\ \vdots \\ \bar{\boldsymbol{x}}_{N+L}^{(N_t-1)} \end{bmatrix}.$$

Before we had $N, N+L$ on the matrix; now we have $N_r N, N_t(N+L)$. The input vector is $N_t(N+L)$: we're stacking the vectors of the input antennas.

All the matrices in H are $N, N+L$ matrices:

$$\begin{bmatrix} \bar{\boldsymbol{H}}_{N,N+L}^{(0,0)} & \bar{\boldsymbol{H}}_{N,N+L}^{(0,1)} & \cdots & \bar{\boldsymbol{H}}_{N,N+L}^{(0,N_t-1)} \\ \bar{\boldsymbol{H}}_{N,N+L}^{(1,0)} & \bar{\boldsymbol{H}}_{N,N+L}^{(1,1)} & \cdots & \bar{\boldsymbol{H}}_{N,N+L}^{(1,N_t-1)} \\ \vdots & \vdots & \ddots & \vdots \\ \bar{\boldsymbol{H}}_{N,N+L}^{(N_r-1,0)} & \bar{\boldsymbol{H}}_{N,N+L}^{(N_r-1,1)} & \cdots & \bar{\boldsymbol{H}}_{N,N+L}^{(N_r-1,N_t-1)} \end{bmatrix}$$

Each of the matrices is like a SISO matrix; we're considering how the information transmitted from antenna j is arriving at antenna i.

In summary, we can express the output of the MIMO system as

$$\bar{\boldsymbol{y}}_{N_rN} = \sqrt{G}\bar{\boldsymbol{H}}_{N_rN,N_t(N+L)}\bar{\boldsymbol{x}}_{N_t(N+L)} + \bar{\boldsymbol{v}}_{N_rN}. \tag{2.95}$$

By the way, L is the **maximum length of the interference channel**; if we have two antennas with a channel with $L_1 = 3$ and another with $L_2 = 7$, we'll use $L = 7$. We'll just put zeros for all the paths which an antenna does not have:

$$h(0), h(1), h(2), 0, 0, 0, 0.$$

If we want to have a well-formed matrix, we select L as the maximum of all the Ls.

The one we got is a compact form, which is the same as the one we had before (with $\bar{\boldsymbol{y}}$), just with bigger dimensions.

By the way, book uses terms *column* and *fat* matrices.

Chapter 5

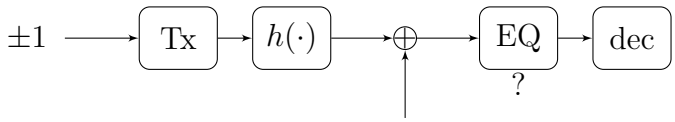
Lesson 25-11-2025

5.1 Linear channel equalization

Ref: *A signal processing perspective* (pp. 57-130), p. 31.

5.1.1 Introduction: super basic block scheme

It's a pretty old topic.
We have a transmitter, then a channel, then a noise which is added **before** anything else.



About magnetic writing; we're writing on a tape, which is noisy; we need to encode before writing.
Fractional sampling: we're sampling at multiples of the Nyquist rate; it's used in order to synchronize.

We don't need to have ±1, it can just be whichever input.

5.1.2 Feedback equalizer

If we have that at a certain time the receive signal is

$$y(0) = h(0)x(0) + h(1)x(-1)\,, \tag{5.1}$$

how do we equalize this?

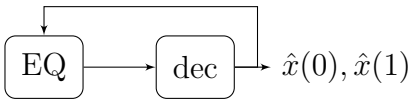
First of all, we'd need to estimate the channel: this means trying to understand the values of $h(0), h(1)$.
If we suppose we estimated $x(-1)$ as $\hat{x}(-1)$, what we do is implement a feedback equalizer: we'll look at

$$\frac{y(0) - h(1)\hat{x}(-1)}{h(0)} \tag{5.2}$$

which will get us $x(0)$.

We can estimate $\hat{x}(-1)$ since we generally have packets: we know the first symbols, thus we can *bootstrap* this machine.

This is the feedback equalizer:



It's also true that if we fail in decoding a symbol, this could be catastrophic: this would lead to a chain of errors.
We need to play with this type of equalization very carefully.

5.1.3 Linear zero forcing equalization (for both SISO and MIMO; only in the z domain); example 2.17

We'll see just something, but if we want to know more we can look at other books.

What we do is that we go in the z domain (see z transform); when we have a sequence, such as $h[0], h[1], h[2], \dots$, we'll write it as

$$\mathbf{h}(z) = h_0 + h_1 z^{-1} + h_2 z^{-2}\,. \tag{5.3}$$

We can revise this by looking at signals and systems.
A delay of n in the time domain corresponds to a multiplication with z^{-n} in the z domain.

We want to find an equalizer $w^*(z)$ such that the combination with $\mathsf{h}(z)$ is just a delay (this is the Heaviside condition):

$$w^*(z)\sqrt{G}h(z) = z^{-\Delta}\,. \tag{2.124}$$

The delay is a degree of freedom which can be used when designing.

We however can see that $w^*(z)$ **might not exist or it might be unstable**.

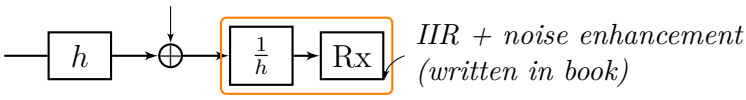
According to the zero-forcing criteria, we'd have

$$w^*(z) = \frac{1}{\mathbf{h}(z)} \cdot \frac{z^{-\Delta}}{\sqrt{G}}\,; \tag{5.4}$$

we first of all ask ourselves: **is this stable?** If it's not stable, the noise coming in might make our system explode!

The system is BIBO if the root of the polynomial $w^*(z)$ is inside the unit circle. The noise, even if bounded, might not be bounded after the system.
This is the problem of the zero forcing; the zero forcing means that we want to just have a single delay; point is that we might have just a delay but with a non-stable filter.

We then ask ourselves: **is the response $w^*(z)$ finite or infinite?** $\rightarrow$ most of the time is **infinite**



We'll then look at the zero forcing **in the MIMO case**. The interesting result is that here for many types of FIR multivariate channel we have FIR zero forcing equalizers, given that we're under certain conditions.

Let's say that the Z-transform of $H[0], \dots, H[L]$ is

$$\mathbf{H}(z) = \sum_{\ell=0}^{L} \mathbf{H}[\ell] z^{-\ell}; \tag{2.129}$$

this implies that the channel will become a polynomial and **we'll look for an equalizer $\mathbf{W}^*(z)$** such that

$$\mathbf{W}^*(z)\sqrt{G}\mathbf{H}(z) = \mathbf{D}(z)\,, \tag{2.130}$$

where

- $\mathbf{D}(z)$ is a diagonal matrix containing possibly different delay terms in the form of $z^{-\Delta}$.

One thing which should be noted is that **we need to have that a left inverse exists for H in order for the equalizer to exist**; if we cannot have a left inverse (for various reasons, such as **H** not being full-rank), then **we won't have an equalizer**.
One condition which is considered already from this moment is that (quoting from the book):

Existence of a left FIR inverse
A MIMO transfer function $\mathbf{H}(z)$ has a left FIR inverse iff $\mathbf{H}(z)$ has full column rank for every complex $z \neq 0$.

Some other conditions will be tackled later, but just as an anticipation, they'll result in saying that the ZF equalizer exists only if

$$N_t(L_{eq} + L + 1) \leq N_r(L_{eq} + 1) \tag{2.146+}$$

What was $H[\ell]$ again?
We know that $\mathbf{H}[\ell]$ is a matrix such that

$$[\boldsymbol{H}[\ell]]_{i,j} = h_{i,j}[\ell]$$

is the channel's response between receiving antenna i and transmitting antenna j. From this, it will follow that

$$[\mathbf{H}(z)]_{i,j} = \mathcal{Z}[h_{i,j}[\cdot]]\, z\,.$$

A signal processing perspective (pp. 57-130), p. 33
As in the SISO case, it is in practice desirable to restrict the focus to FIR equalizers, meaning that each entry of $\mathbf{w}(z)$ corresponds to an FIR filter of order L_{eq}. Do exact left inverses of $\mathbf{H}(z)$ exist? Yes, as it turns out. More precisely, the answer is found in what is called **perfect recoverability**.

So, if the *perfect recoverability* holds, we'll be sure to find left inverses of the $\mathbf{H}$ matrix; we need the left inverse since it will be used when we'll determine the ZF equalizer.

Here in **MIMO FIR filters may exist!** Before, in SISO, **they might have not!**

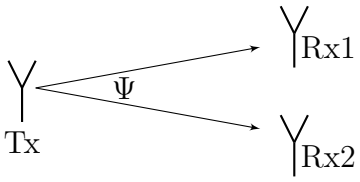
About matrices: until now we saw that the inverse of whichever matrix $\boldsymbol{A}$ is such that $\boldsymbol{A}\boldsymbol{A}^{-1} = \boldsymbol{I}$ and $\boldsymbol{A}^{-1}\boldsymbol{A} = \boldsymbol{I}$; this only holds if it's square; generally, the left and right inverses are different.

About this paragraph
The notation we wrote here just gives an idea; everything will be treated in the time domain.

Example 2.17

For $N_t = 1$ and $N_r = 2$, find the conditions on $\mathbf{H}(z)$ such that FIR ZF equalizers exist.

If we have 1 transmit antenna and 2 receive antennas; in general we might transmit to both antennas:



> Digital always includes a delay!

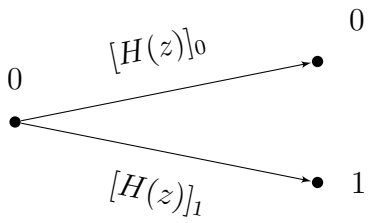
We then need to decide what to do; **we might think about selecting the stronger signal**; one might have a reflection and such; this is called **antenna selection**.
It's easy to measure the amplitude of a signal, but we can't know whether the signal is clean.
Someone came up with something called **MRC (Max Ratio Combining): we weight each signal with the complex conjugate**; this allows us **to have the best SNR**. This is still done, even with analog transmissions.
What we have is that we want to satisfy

$$[\mathbf{W}(z)]_0^* \sqrt{G}\,[\mathbf{H}(z)]_0 + [\mathbf{W}(z)]_1^* \sqrt{G}\,[\mathbf{H}(z)]_1 = z^{-\Delta}\,. \tag{5.5}$$

$\mathbf{H}(z)$ is a column vector:

↓ *read example (continued on next page)*

Example 2.17 (continued)



$[H(z)]_0$ might be $= h_{0,0} + h_{0,1}z^{-1} + \dots$
Anyways, we'll have

$$\mathbf{H}(z) = \begin{bmatrix} [H(z)]_0 \\ [H(z)]_1 \end{bmatrix}. \tag{5.6}$$

The only way for this vector to not be full rank is if $\mathbf{H}(z) = \mathbf{0}$ for some z.
If we have that there exists z^* such that $\mathbf{H}(z^*) = \mathbf{0}$ and this happens only if the polynomials have a common root, which is z^*; we don't have full rank only if those two polynomials share a common root!
If those polyomials are coprime (they don't share a common root!), then it's possible to find an exact FIR inverse.

We care so much about the full rank since a left inverse only exists if $\mathbf{H}(z)$ has full column rank for every $z \neq 0$.
If we have

$$\mathbf{H}(z) = \begin{bmatrix} 1 + z^{-1} \\ 1 + z^{-1} \end{bmatrix},$$

aka the two are equal, we can't find FIR; if we have a half, then it's possible!

$$\mathbf{H}(z) = \begin{bmatrix} 1 + \frac{z^{-1}}{2} \\ 1 + z^{-1} \end{bmatrix}.$$

This is why it's not possible for SISO systems.

5.2 Moore-Penrose pseudo-inverse

In the future, we'll need another version of an inverse; this is the Moore-Penrose pseudo-inverse.
This can be applied to non-square matrices!

The pseudoinverse $\boldsymbol{A}^\dagger$ of an $N \times M$ matrix $\boldsymbol{A}$ is the unique matrix satisfying

$$\boldsymbol{A}\boldsymbol{A}^\dagger\boldsymbol{A} = \boldsymbol{A} \tag{B.30}$$
$$\boldsymbol{A}^\dagger\boldsymbol{A}\boldsymbol{A}^\dagger = \boldsymbol{A}^\dagger \tag{B.31}$$

and also such that $\boldsymbol{A}\boldsymbol{A}^\dagger$, $\boldsymbol{A}^\dagger\boldsymbol{A}$ are both Hermitian.

Taking the already known concept of inversion, we have that:

- If $\boldsymbol{A}^*\boldsymbol{A}$ is invertible, then

$$\boldsymbol{A}^\dagger = (\boldsymbol{A}^*\boldsymbol{A})^{-1}\boldsymbol{A}^* \tag{B.32}$$
$$\boldsymbol{A}^\dagger\boldsymbol{A} = \boldsymbol{I}\,; \tag{5.7}$$

  this may be the case if $N \geq M$ (tall matrix).

- If $\boldsymbol{A}\boldsymbol{A}^*$ is invertible, then

$$\boldsymbol{A}^\dagger = \boldsymbol{A}^*(\boldsymbol{A}\boldsymbol{A}^*)^{-1} \tag{B.33}$$
$$\boldsymbol{A}\boldsymbol{A}^\dagger = \boldsymbol{I}\,; \tag{5.8}$$

  this may be the case if $N \leq M$ (fat matrix).

- If $\boldsymbol{A}$ is square and invertible, then (B.32), (B.33) coincide and the pseudoinverse reduces to the regular inverse,

$$\boldsymbol{A}^\dagger = \boldsymbol{A}^{-1}\,.$$

⚠ The right inverse and the left inverse are not the same!
⚠ This only holds if the matrix is **full-rank** (which is then the reason why we ask that the $\boldsymbol{H}$ matrix is full-rank).
We'll use this a lot in the course.

Another point is that if we remember the **singular value decomposition**, then we have

$$\boldsymbol{A} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^*$$
$$\boldsymbol{A}^\dagger = \mathbf{V}\boldsymbol{\Sigma}^{-1}\mathbf{U}^*\,.$$

By the way, we can do this in matlab using the `pinv()` function.

5.3 Actually finding the MIMO zero forcing equalizer

Let's write the problem in the time domain, going from

$$\mathbf{W}^*(z)\sqrt{G}\mathbf{H}(z) = \mathbf{D}(z)\,, \tag{2.130}$$

where $\mathbf{D}(z)$ is made of diagonal elements:

$$\mathbf{D}(z) = \begin{bmatrix} z^{-\Delta_1} & & 0 \\ & z^{-\Delta_2} & \\ 0 & & \ddots \end{bmatrix}$$

to

$$\sqrt{G}\sum_{\ell=0}^{L_{eq}} \mathbf{W}^*[\ell]\mathbf{H}[n-\ell] = \mathrm{diag}[\delta[n-\Delta_0], \ldots, \delta[n-\Delta_{N_t-1}]]\,. \tag{2.132}$$

where:

- $\mathbf{W}$ is a $N_r \times N_t$ matrix (I deduced the N_t from the fact that later the columns are indexed with j).

- $\mathbf{H}$ is a $N_r \times N_t$ matrix. *(Note: usually H_{ij} means from j to i).*

This is a diagonal matrix with delays since we designed our equalizer to establish so.

We've allowed ourselves to have different delays ($\Delta_n, n \in [0, N_t - 1]$); finding the zero forcing equalizer means solving the following equation for the stream j:

$$\sqrt{G}\sum_{\ell=0}^{L_{eq}} \mathbf{w}_j^*[\ell]\mathbf{H}[n-\ell] = \mathbf{d}_j[n] \quad n = 0, 1, \dots, \underbrace{L_{eq} + L}_{\rightarrow \text{ consequence of convolution}} \quad ; \tag{2.133}$$

It makes perfect sense to now ask ourselves what all the parameters are, specifically; let's see them in detail.

$\mathbf{w}_j[\ell]$
$\mathbf{w}_j[\ell]$ is a $N_r \times 1$ column vector such that

$$\mathbf{w}_j[\ell] = \mathbf{W}[\ell]_{\cdot,j} : \begin{bmatrix} w_{1,j}[\ell] \\ \vdots \\ w_{N_r,j}[\ell] \end{bmatrix}.$$

It represents the equalization applied to the signals transmitted by the antenna j towards all the receiving antennas.

$\mathbf{d}_j[n]$
With reference to the equation (2.133) we define $\mathbf{d}_j[n]$ as the **target delays for transmitted signal** j, with $j = 0, \dots, N_t - 1$; it will be a $N_t \times 1$ vector made of **all zeros** and a $\delta[n - \Delta_j]$ **at position** j, where $\Delta_j \in [0, \dots, L_{eq} + L]$ is the target delay for the transmitted signal j.

We can formulate (2.133) in vector notation as

$$\sqrt{G}\bar{\mathbf{w}}_j^*\mathcal{T} = \bar{\mathbf{d}}_j\,, \tag{2.137}$$

with the various elements being as described in the following:

$\bar{\mathbf{w}}_j$

$\bar{\mathbf{w}}_j$ is a $N_r(L_{eq}-1)\times 1$ vector obtained by stacking all $\mathbf{w}_j$ vectors on top of one another, as

$$\bar{\mathbf{w}}_j = \begin{bmatrix} \mathbf{w}_j[0] \\ \vdots \\ \mathbf{w}_j[L_{eq}] \end{bmatrix}, \tag{2.134}$$

embedding all the equalizer responses over the delay dimension, always referring to transmit antenna j.

$\bar{\mathbf{d}}_j$

We then define $\bar{\mathbf{d}}_j$ as a $N_t(L_{eq}+L+1)\times 1$ vector:

$$\bar{\mathbf{d}}_j = \begin{bmatrix} \mathbf{d}_j[0] \\ \vdots \\ \mathbf{d}_j[L_{eq}+L] \end{bmatrix}; \tag{2.135}$$

discussing the positions at which this vector is non-zero deserves its own discussion. Due to how $\mathbf{d}_j[n]$ is defined, we know that it will have a 1 only at position j and only if $n=\Delta_j$. Let's now consider $\bar{\mathbf{d}}_j$ for a simple case such as $N_t=2, j=0, L_{eq}+L=1$; we'll have

$$\bar{\mathbf{d}}_0 = \begin{bmatrix} \mathbf{d}_0[0] \\ \mathbf{d}_0[1] \end{bmatrix} = \begin{bmatrix} \begin{bmatrix} \delta[0-\Delta_0] \\ 0 \end{bmatrix} \\ \begin{bmatrix} \delta[1-\Delta_1] \\ 0 \end{bmatrix} \end{bmatrix};$$

in this simple example we see that the various subvectors are located at multiples of N_t, specifically at $nN_t, n\in[0, L_{eq}+L]$; in each subvector, the δ is located at the j-th element. We'll thus have the various deltas at positions

$$N_t n + j, \quad n = 0, \dots, L_{eq}+L$$

where j is fixed by the vector $\bar{\mathbf{d}}_j$ we're considering.
It's also true that the δs will only be 1 if $n=\Delta_j$ and we can thus simplify the previous equation by saying that we'll have 1 in the vector at positions

$$N_t\Delta_j + j,$$

which of course can only be satisfied for those Δ_j which are $\Delta_j \leq L_{eq}+L$.

Toeplitz matrix $\bar{\mathcal{T}}$

The Toeplitz matrix $\bar{\mathcal{T}}$ is defined as

$$\bar{\mathcal{T}} = \begin{bmatrix} \mathbf{H}[0] & \dots & \mathbf{H}[L] & \mathbf{0} & \dots & \mathbf{0} \\ \mathbf{0} & \mathbf{H}[0] & \dots & \mathbf{H}[L] & \mathbf{0} & \mathbf{0} \\ \vdots & & \ddots & & \ddots & \vdots \\ \mathbf{0} & \dots & \mathbf{0} & \mathbf{0} & \mathbf{H}[0] & \dots \mathbf{H}[L] \end{bmatrix}, \tag{2.136}$$

having dimensions of $N_r(L_{eq}+1) \times N_t(L_{eq}+L+1)$.

We reach a linear system of equations → under determined means infinite sol, over determined means no-sol.

We then consider that we're trying to minimize the error in (2.137), which is proportional to

$$\|\sqrt{G}\bar{\mathbf{w}}_j^* \bar{\mathcal{T}} - \bar{\mathbf{d}}_j^*\|^2 . \quad \textit{(RIP Tomba †)} \tag{2.138-ish}$$

We can show (*A signal processing perspective* (pp. 57-130), p. 35) that (2.138-ish) can be minimized using the left Moore-Penrose pseudo-inverse of the matrix $\bar{\mathcal{T}}^*$, aka $(\bar{\mathcal{T}}^*)^\dagger$:

$$\bar{\mathbf{w}}_j^{ZF} = \frac{1}{\sqrt{G}}(\bar{\mathcal{T}}^*)^\dagger \bar{\mathbf{d}}_j \quad \text{time domain} \tag{2.145}$$

$$\mathbf{W}^{ZF} = \frac{1}{\sqrt{G}}(\bar{\mathcal{T}}^*)^\dagger \bar{\mathbf{D}} \quad \text{z domain} \tag{2.146}$$

We can see how we can find the zero-forcing solution for MIMO.

I'd like to elaborate a lil bit on this; we essentially have that

$$\sqrt{G}\mathbf{W}^{ZF*}\mathcal{T} = \mathbf{D}^* \implies \mathbf{W}^{ZF*} = \frac{1}{\sqrt{G}}\mathbf{D}^*(\mathcal{T})^\dagger .$$

Now we need to do the complex transpose of that, in order to find the equalizer. Before doing that, it's relevant to know that

$$(\bar{\mathcal{T}}^\dagger)^* \overset{\text{tall}}{=} [(\boldsymbol{A}^*\boldsymbol{A})^{-1}\boldsymbol{A}^*]^* = \boldsymbol{A}(\boldsymbol{A}^*\boldsymbol{A})^{-1} = (\bar{\mathcal{T}}^*)^\dagger ;$$

then we see that the complex transpose of the equation up there does indeed give (2.146).

z domain?

I think it was professor to write "z domain"; I'm positive this whole thing goes for the time domain!

For a frequency flat channel, this simplifies to

$$\mathbf{W}^{ZF} = \frac{1}{\sqrt{G}}(\mathbf{H}^\dagger)^* .$$

Just as a sidenote, this equation gives what some exercises refer to as the *perfect equalizer*.

One important thing which was overlooked (until here) is that this comes as a solution to (2.137); thing is that **said solution only exists if**

$$N_t(L_{eq}+L+1) \leq N_r(L_{eq}+1) \tag{2.146+}$$

(aka **only exists if $\bar{\mathcal{T}}$ is either square or tall**).

$\mathbf{W}^{ZF}$

The $\mathbf{W}^{ZF}$ matrix is defined as

$$\mathbf{W}^{ZF} = [\bar{\mathbf{w}}_0^{ZF} \quad \dots \quad \bar{\mathbf{w}}_{N_t-1}^{ZF}],$$

in a way such that the j-th column represents the equalizer coefficients for the jth transmitted signal in a way such that the values from kN_r to $(k+1)N_r, k \in \mathbb{Z}$ will represent the coefficients used to equalize at the kth receiver.

$\bar{\mathbf{D}}$

Matrix $\bar{\mathbf{D}}$ is defined as

$$\bar{\mathbf{D}} = [\bar{\mathbf{d}}_0 \quad \dots \quad \bar{\mathbf{d}}_{N_t-1}],$$

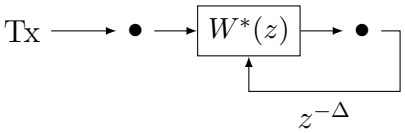
which, due to what has been said before, will be zero except for positions

$$(N_t\Delta_j + j, j),$$

which will be 1, if the position exists in the matrix.

5.4 MIMO Degrees of Freedom and ZF Equalization

Let's conceptually **look at a problem which arises only when we look at SISO** due to the lack of degrees of freedom. If we have 3 transmitter and 4 receivers, we can introduce delays to make everything look like just one delay. If we just have 1 receiver, we can't make everything look just like one delay, though:



For the first antenna, we modify $W^*(z)$ so that it can transform the first channel response into a delay $z^{-\Delta}$; the other channel response will not be such that the same $W^*(z)$ will be able to enforce the same delay as a result (at least if the response is different, which is generally the case): we've already used the only degree of freedom we had by setting Δ.
Rem. *In the MIMO case, we have more degrees of freedom and that's why we generally have a solution, provided that* $\mathbf{H}$ *is full rank.*

Example 2.18

Compute the ZF equalizer for a frequency-flat channel with $N_r \geq N_t$.

In a frequency flat channel we can see that the matrix is just a pseudo inverse of $\mathbf{H}$, since we have $T = \mathbf{H} \rightarrow$ delay of L. Then do not $L_{eq} \rightarrow \infty$ (minimize eq. error). We'll thus have:

$$W^{ZF} = \frac{1}{\sqrt{C}}\mathbf{H}^{+} \tag{5.9}$$

There's a difference between the SISO and the MIMO cases.

- **SISO:** we have the **problem of stability** and even for simple channels we might **not have a causal filter**.
- **MIMO:** The **perfect equalizer** of a **FIR channel is FIR**.

Example 2.19

Verify that the W^{ZF} derived in Example 2.18 effects perfect ZF equalization.

Solution

The application of the equalizer to the channel gives:

$$W^{ZF}\sqrt{C}\mathbf{H} = (\mathbf{H}\mathbf{H}^{+})^{-1}\mathbf{H}^{H}\mathbf{H} \implies \mathbf{I} \quad \text{Identity} \tag{5.10}$$

If we ignore the noise, we have that the channel behaves like an Identity.

If we have 4 antennas in transmission and reception, it's like if we had 4 parallel channels: this is a big step forward.

5.5 LMMSE equalization

Until now **we ignored noise**: the big problem with zero forcing which does not disappear with MIMO is that **noise generally gets amplified**.

If we multiply the w with y, we get an error on top of the delay. We'll introduce a new concept, aka **trying to minimize the noise**. We're not just inverting the channel: we now have another target; we're trying to invert the channel as much as we can while also limiting the noise as much as we can.

ZF equalization neglects the noise: we instead would like to **take the noise into account**; in order to do so, we use the MMSE metric we already introduced. Our equalizer

will then be a MMSE estimator and since we stay in the context of linear equalization, it will be a Linear MMSE estimator (LMMSE).

In this section we'll try to derive a LMMSE estimator using the theoretical concept we defined for MMSE estimation.

For this derivation we consider both $\mathbf{x}[n], \mathbf{s}[n]$ as **wide sense stationary**, using the notations:

$$R_x[l] = E[\mathbf{x}[n]\mathbf{x}^*[n+l]] \tag{5.11}$$
$$R_x[l] = N_0\delta[l] \cdot I \tag{5.12}$$

(Pass on this / get to some)

It can be shown that thanks to the transmit-receive relationships we defined:

$$R_y[l] = G\sum_{i=0}^{L}\sum_{j=0}^{L}\mathbf{H}[i]R_x[l+i-j]\mathbf{H}^*[j] + N_0\delta[l]I \tag{5.13}$$

$$R_{yx}[l] = \sqrt{G}\sum_{i=0}^{L}\mathbf{H}[i]R_x[l+i] \tag{5.14}$$

We want to look at the difference between $\mathbf{x}[n-\Delta]$ (which is the desired signal) and the equalized output (obtained by convolving the output y $(N_r \times 1)$ and the equalizer W $(N_t \times N_r)$). We need to remember that y will have the noise in it:

$$\hat{x}[n] - x[n-\Delta] = \sum_{l=0}^{L_{eq}}\mathbf{W}^*[l]\mathbf{y}[n-l] \tag{5.15}$$

> **⚠ Totally not trivial implication**
>
> Differently from what we have in the original treatment, here we're not considering a frequency-flat channel. Rather, this channel has time dependency: for this reason, we're considering the convolution between $\mathbf{W}$ and y.

We want to minimize this difference here! We can also obtain a matricial notation by stacking the various vectors/matrices; we'll obtain:

$$\hat{\mathbf{x}} = x[n-\Delta] - \mathbf{W}^*\vec{y}[n] \tag{5.16}$$

This is good to find the LMMSE estimator (*in the book are all the details, professor does not really care about them*).

Rem. *The book never states this explicitly, but those matrices are:*

- $W^* : N_t \times N_r(L_{eq}+1)$
- $y[n] : N_r(L_{eq}+1)$

And they're obtained through a simple stacking.

Although neither the book or professor were really clear about this, I'd say that our equalizer is designed so that

$$\mathbf{W}^* y[n] \approx x[n-\Delta] \tag{5.17}$$

The problem is that ZF equalizers amplify the noise; we'd like to have a compromise between equalization of the channel and non-amplification of the noise.

We thus form the W^{MMSE} estimator following from **The estimator is simply**

$$\mathbf{W}^{LMMSE} = \mathbf{R}_y^{-1}\mathbf{R}_{yx} \tag{5.18}$$

which in this case (that of *WSS signals/noise*) yields

$$W^{MMSE} = \mathbf{R}_{\vec{y}[n]}^{-1}\mathbf{R}_{\vec{y}[n]x[n-\Delta]} \tag{5.19}$$

$$= \begin{bmatrix} R_y[0] & R_y[1] & \dots & R_y[L_{eq}] \\ R_y[1] & R_y[0] & \dots & R_y[L_{eq}-1] \\ \vdots & \vdots & \ddots & \vdots \\ R_y[L_{eq}] & R_y[L_{eq}-1] & \dots & R_y[0] \end{bmatrix}^{-1} \begin{bmatrix} R_{yx}[-\Delta] \\ R_{yx}[1-\Delta] \\ \vdots \\ R_{yx}[L_{eq}-\Delta] \end{bmatrix} \tag{5.20}$$

We then have that from

$$E = \mathbb{E}[(\mathbf{x}-\hat{x})\mathbf{y}^H(\mathbf{x}-\hat{x})^H] \tag{5.21}$$

$$= \mathbf{R}_x - \mathbf{R}_{yx} - \mathbf{R}_{yx}^H + \mathbf{R}_y \tag{5.22}$$

$$= \mathbf{R}_x - \mathbf{W}^H\mathbf{R}_y^{-1}\mathbf{R}_{yx} - \dots \tag{5.23}$$

we can find the MMSE matrix by considering the various matrices we've already found, which will specifically give:

$$E = \mathbb{E}[\tilde{x}[n]\tilde{x}^*[n]] \tag{5.24}$$

$$= R_x[0] - R_{yx[n]y[n]}^H R_{y[n]}^{-1} R_{yx[n]x[n-\Delta]} \tag{5.25}$$

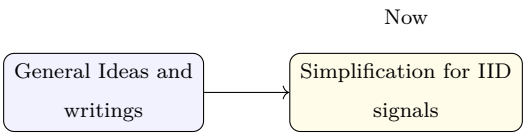
We don't need to remember the manipulations which are involved.
There won't be exercises about this!

Exam

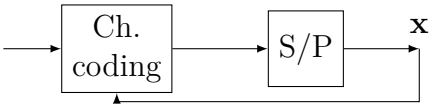
We can bring 1A4 paper, with printings on only one side. We'll be able to use them for everything but not for the quizzes!!

Warning

NEVER forget the noise, since it's the most important thing here.



If, in order to simplify things, we write as follows:



We have that the values of $\mathbf{x}$ are actually dependent, since the channel coding introduces dependencies and redundancy.

We make an **(incorrect!!)** assumption which is that the vector $\mathbf{x}$ is made of **IID symbols**; this assumption allows us to have a simple writing of $\mathbf{R}_x$:

$$R_x[l] = \frac{E_x}{N_t}\delta[l]I \tag{5.26}$$

If we choose E_x in a specific manner, the R_x matrix becomes an identity, which simplifies things by a lot. This won't work in real systems, tho!

We'll obtain that the MMSE matrix can be written as:

$$\begin{aligned}
\tilde{W}^{MMSE} &= \mathbf{R}^{-1}_{\vec{y}[n]}\mathbf{R}_{\vec{y}[n]x[n-\Delta]} && (5.27)\\
&= \frac{1}{\sqrt{G}}\left(\mathcal{T}^H\mathcal{T} + \frac{N_t}{\mathrm{SNR}}\mathbf{I}\right)^{-1}\mathcal{T}^H D && (5.28)\\
&= \frac{1}{\sqrt{G}}\mathcal{T}^H\left(\mathcal{T}\mathcal{T}^H + \frac{N_t}{\mathrm{SNR}}\mathbf{I}\right)^{-1} D && (5.29)
\end{aligned}$$

The relevant thing is the SNR: if we push the SNR to $+\infty$, we have that the noise is disappearing and we'll find again the zero forcing equalizer! It makes sense that if the noise is disappearing the noise equalizer and the zero-forcing equalizer converge to be the same.

This will never be asked, but for completeness:

The $\mathcal{T}$ matrix is

Toeplitz matrix $\mathcal{T}$

The Toeplitz matrix $\mathcal{T}$ is defined as

$$\mathcal{T} = \begin{bmatrix} H[0] & \dots & H[L] & 0 & \dots & 0 \\ 0 & H[0] & \dots & H[L] & \dots & 0 \\ \vdots & & \ddots & & \ddots & \vdots \\ 0 & \dots & 0 & H[0] & \dots & H[L] \end{bmatrix} \tag{5.30}$$

having dimensions of $N_r(L_{eq}+1) \times N_t(L_{eq}+L+1)$.

and D is all zeros except for

$$[D]_{\Delta N_t:\Delta N_t+N_t-1,:} = \mathbf{I} \tag{5.31}$$

The alternative form is obtained using the **Matrix inversion lemma**. We know that this is a tool which is very much used.

What's important now is **the error**, which will be:

$$E = \frac{E_x}{N_t}\mathbf{I} - \frac{E_x}{N_t} D^T \mathcal{T}^* \left(\mathcal{T}\mathcal{T}^* + \frac{N_t}{\text{SNR}}\mathbf{I} \right)^{-1} \mathcal{T} D \tag{5.32}$$

Note: *EXAM TYPE OF QUESTION: demonstrate that G goes to zero if nr goes to infty*

If we **push the SNR to infinity**, we again have that the whole second term becomes $(E_x/N_t)I$ and thus we'd have $E \to 0$: we're again in the zero-forcing situation.

We didn't calculate the error in case of zero forcing; we'll see that the error is actually worse than this one.

About the inversion of a matrix of matrices

We can correctly invert a matrix by finding its determinant:

$$A = \begin{bmatrix} d_{11} & d_{12} \\ d_{21} & d_{22} \end{bmatrix} \tag{5.33}$$

$$\det(A) = d_{11}d_{22} - d_{12}d_{21} \tag{5.34}$$

Instead:

$$\mathcal{A} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \tag{5.35}$$

We'll have

$$\det(\mathcal{A}) = \det A_{11} \det A_{22} \tag{5.36}$$
$$- \det A_{12} \det A_{21} \tag{5.37}$$

We can find this as reference 125 in the book.

✖ Recommended read: example 2.21

Example 2.21, which was mentioned by professor many times, uses (2.163) together with some other smart tricks in order to obtain the LMMSE estimator for frequency flat channels, using the precoding writing of the signal,

$$x[n] - \sqrt{\frac{E_s}{N_t}} F[s]s[n] \tag{5.38}$$

, also assuming that the $s[n]$ symbols are IID, strict-sense stationary and unit-variance: this, together with the imposition that $\Delta = 0$, allows us to write that

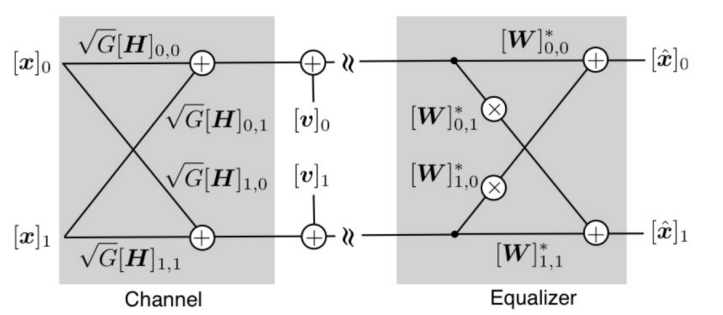
$$\mathbf{R}_x[0] = \frac{E_s}{N_t}\mathbb{E}[\mathbf{F}[n]\mathbf{s}[n]\mathbf{s}^*[n]\mathbf{F}^*[n]] = \frac{E_s}{N_t}\mathbf{F}[n]\mathbf{F}^*[n]$$

(which is admittedly a bit of a stretch, since the book just said that $\mathbf{R}_x[0] = \mathbf{F}\mathbf{F}^*$, assuming that all symbols are precoded through the same precoder and that [0] expresses a delay of 0).
In turn, this eventually allows us to write

$$\mathbf{W}^{\text{MMSE}} = \frac{1}{\sqrt{G}} \left(\mathbf{H}\mathbf{F}\mathbf{F}^*\mathbf{H}^* + \frac{N_t}{\text{SNR}}\mathbf{I} \right)^{-1} \mathbf{H}\mathbf{F}, \tag{2.182}$$

for which we then have

Normally asked question: MIMO Eq



What is the length of the equalizer?

$L_{eq} = 2?$ $1?$ **No.**

L_{eq} is **zero** because we don't have memory. **We have only a matrix.**

Convention:

indices: $\boxed{0}, 1, \ldots, L_{eq}$

$\underbrace{\qquad}_{L_{eq}+1}$